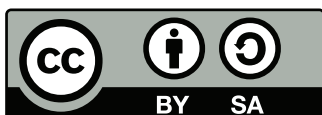


CardioMechanics

User's Manual & Documentation

Version 1.1

Dr.-Ing. Tobias Gerach
July 27, 2026



This document - excluding the cover, pictures, tables and graphs - is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0): <https://creativecommons.org/licenses/by-sa/4.0/>

Contents

1	Introduction	6
2	Getting Started	6
2.1	System Requirements	6
2.2	Installation Instructions	7
3	Mathematical Model	7
3.1	Modeling Cardiac Mechanics	7
3.2	Spatial Discretization	9
3.3	Time Discretization	10
3.4	The Inverse Problem: Active Stress Estimation	11
3.5	Units	13
4	File Formats	13
4.1	Input Files	13
4.1.1	CardioMechanics Config File (*.xml)	13
4.1.2	Element file (*.ele)	14
4.1.3	Node File (*.node)	14
4.1.4	Surface Elements File (*.sur)	15
4.1.5	Fiber Orientation File (*.bases)	15
4.1.6	Local Activation Time File (*.txt)	15
4.1.7	Target Surface Files (*.list, *.dat)	16
4.2	Output Files	16
4.2.1	VTK Unstructured Grid Files (*.vtu)	16
4.2.2	ParaView Data File (*.pvd)	16
4.3	acCELLerate Specific Input Files	16
4.3.1	Config File (*.acft)	17
4.3.2	Condition File (*.acnd)	18
4.3.3	Cell Model File (*.cmf)	18
4.3.4	Material File (*.def)	18
4.3.5	Sensor File (*.txt)	19
4.3.6	CellModel Parameter File (*.ev)	19
4.3.7	ForceModel Parameter File (*.fv)	20
5	Simulation Framework	20
5.1	Mesh	23
5.2	Materials	24
5.2.1	Constitutive Models	25
5.2.2	Tension Models	29
5.3	Solver	35
5.4	ActiveStressEstimator (Inverse Problem)	36
5.5	Solver Plugins	38
5.5.1	LoadUnloadedState	39
5.5.2	ContactHandling	39
5.5.3	PointsCtrl	41

5.5.4	Circulation	41
5.5.5	ApplyPressure	51
5.5.6	ApplyPressureFromFunction	51
5.5.7	ApplyPressureFromFunctionNodeExport	53
5.5.8	RobinBoundary	53
5.5.9	RobinBoundaryGeneral	54
5.5.10	ReferenceRecovery	54
5.5.11	acCELLerate	56
5.6	Export	59
6	Tools	60
6.1	BidomainMatrixGenerator	60
6.2	PETScVec2VTK	62
6.3	ConvertT4toT10	62
6.4	VTPExtractSurfaceFromMesh	63
6.5	ExtractSurfaceNodesFromVTU	63
6.6	FixNodes	63
7	Scripts	64
7.1	VTK2tetgen	64
7.2	CreateTargetSurfaces	64
7.3	ConvertPETScDir2VTK	65
7.4	tuneCV	65
7.5	CircWholeHeart	66
7.6	createSimDir	66
7.7	ClipVTUwithVTU	67

Listings

1	CardioMechanics Config File (*.xml)	13
2	Element file (*.ele)	14
3	Node File (*.node)	14
4	Surface Elements File (*.sur)	15
5	Fiber Orientation File (*.bases)	15
6	Local Activation Time File (*.txt)	15
7	Target surface list file (*.list)	16
8	Target surface data file (*.dat)	16
9	acCELLerate configuration file (*.acft)	17
10	acCELLerate condition file (*.acnd)	18
11	acCELLerate cell model file (*.cmf)	18
12	acCELLerate material file (*.def)	18
13	acCELLerate material file example	19
14	acCELLerate sensor file (*.txt)	19
15	Ionic model parameter file (*.ev)	19
16	.xml settings for mesh initialization	23
17	.xml settings for material definitions	24
18	.xml settings for Guccione material	26
19	.xml settings for general Rivlin material	27
20	.xml settings for Holzapfel material	27
21	.xml settings for Usyk material	28
22	.xml settings for NeoHooke material	29
23	File structure of a tension.list file	29
24	File structure of individual tension files	29
25	.xml settings for the File tension model	29
26	.xml settings for the Lumens tension model	30
27	.xml settings for the FromFunction tension models	31
28	.xml settings for the Land17 tension model	32
29	.xml settings for the TanH tension model	33
30	.xml settings for the Bestel tension model	34
31	.xml settings for the TensionEstimator tension model	34
32	.xml settings for the solver class	36
33	.xml settings for the active stress estimator	37
34	.xml settings for solver plugins	38
35	.xml settings for LoadUnloadedState plugin	39
36	.xml settings for ContactHandling plugin	40
37	.xml settings for the PointsCtrl plugin	41
38	.xml settings for the circulation plugin	42
39	.xml settings for the coupling damping of the circulation plugin (subkeys of <Circulation>)	44
40	.xml settings for the preloading procedure of the circulation plugin (subkeys of <Circulation>)	44
41	.xml settings for a limit cycle check for all circulation models (subkey of <Circulation><Circs><Circ_X>)	45
42	.xml settings for the 4-chamber circulation model	47

43	.xml settings for the 1-chamber circulation model	49
44	.xml settings for the 2-chamber circulation model	50
45	.xml settings for the ApplyPressure plugin	51
46	.xml settings for the ApplyPressureFromFunction plugin	52
47	.xml settings for the ApplyPressureFromFunctionNodeExport plugin	53
48	.xml settings for the RobinBoundary plugin	54
49	.xml settings for the RobinBoundaryGeneral plugin	54
50	.xml settings for the ReferenceRecovery plugin	55
51	.xml settings for the acCELLerate plugin	58
52	.xml settings for data export options.	60
53	Runtime options for BidomainMatrixGenerator	61
54	Syntax for PETScVec2VTK	62
55	Syntax for ConvertT4toT10	62
56	Syntax for VTPExtractSurfaceFromMesh	63
57	Syntax for ExtractSurfaceNodesFromVTU	63
58	Syntax for FixNodes	63
59	Syntax for VTK2tetgen.py	64
60	Syntax for CreateTargetSurfaces.py	64
61	Syntax for ConvertPETScDir2VTK.py	65
62	Syntax for tuneCV.py	65
63	Syntax for createSimDir.py	67
64	Syntax for ClipVTUwithVTU.py	67

1 Introduction

CardioMechanics is a finite element analysis software for biomechanics problems specifically developed for cardiac mechanics at the [Institute of Biomedical Engineering, Karlsruhe Institute of Technology \(KIT\)](#). It supports passive and active tissue mechanics for heart geometries with 1-, 2-, or 4-chambers. The use of a flexible plugin interface gives access to additional functionality such as the recovery of the reference configuration, passive parameter optimization, circulatory system coupling, and coupling to a solver for cardiac electrophysiology. A direct interface to the now retired cardiac electrophysiology simulator *acCELLerate* developed by Gunnar Seemann et al. is provided but coupling to other software packages is possible if you want to put in the work required.

2 Getting Started

CardioMechanics is the main component of this software package. To make electromechanical simulations possible, the additional components *acCELLerate* and *CellModelTest* (with subcomponents *ElphyModelTest* and *ForceModelTest*) were added and can be utilized via the plugin interface of *CardioMechanics*. Currently, no precompiled version is available and the entire software has to be build from source. Make sure to follow the build instructions ([docs/BUILD.md](#)) to install from source using CMake or read the following section.

2.1 System Requirements

The following requirements have to be installed before trying to build *CardioMechanics* from source. We recommend using a package manager (e.g. [Homebrew](#) on macOS or `apt` on Ubuntu) whenever possible.

- C and C++ compilers (e.g. `gcc/g++` or `clang/clang++`)
- [CMake](#) ≥ 3.20
- [pkg-config](#)
- [zlib](#)
- [gfortran](#)
- [git](#)
- [PETSc](#) (built with `--prefix` so that a `PETSc.pc` pkg-config file is generated)
- [VTK](#) ≥ 9
- [Open MPI](#)
- [Python3](#) (optional, for the tools in `tools/python/`)

The `docker/` directory contains the Dockerfiles used by CI: `Dockerfile-thirdparty-petsc` source-builds [PETSc](#) with the tested version, while [Open MPI](#) and [VTK](#) are installed from `apt` (`libopenmpi-dev`, `libvtk9-dev`). They can serve as a reference for building the dependencies manually.

2.2 Installation Instructions

Only PETSC_DIR (and optionally PETSC_ARCH) are required so that CMake can locate the PETSc pkg-config file. Add them to your shell configuration file (e.g. .bashrc or .zshrc):

```
1 export PETSC_DIR=/path/to/petsc      # prefix install directory
2 export PETSC_ARCH=                  # leave empty if PETSc was installed
   with --prefix
```

If PETSC_ARCH is empty, the pkg-config file is expected at \$PETSC_DIR/lib/pkgconfig/PETSc.pc; if it is non-empty, at \$PETSC_DIR/\$PETSC_ARCH/lib/pkgconfig/PETSc.pc. VTK and Open MPI are found via CMake's standard search paths; on macOS with Homebrew no extra variables are needed.

Configure and build using the provided presets (binaries are placed in _build/<preset>/bin/):

```
1 cmake --preset release
2 cmake --build --preset release -j
```

Use the debug preset in the same way for a debug build. To build without presets:

```
1 cmake -S . -B _build/release -DCMAKE_BUILD_TYPE=Release
2 cmake --build _build/release -j
```

Finally, add the executables and Python tools to your PATH variable:

```
1 export PATH="$PATH:/path/to/CardioMechanics/_build/release/bin"
2 export PATH="$PATH:/path/to/CardioMechanics/tools/python"
```

Running the binaries directly from _build/<preset>/bin/ is sufficient. To install them elsewhere, use cmake --install, which copies the executables into <prefix>/bin/:

```
1 cmake --install _build/release --prefix
   /path/to/install/CardioMechanics.opt
```

Always pass --prefix (or pin CMAKE_INSTALL_PREFIX per preset). Without it, CMAKE_INSTALL_PREFIX defaults to /usr/local on both Linux and macOS, and installing there requires root.

See [docs/BUILD.md](#) for pinning PETSC_DIR and the install prefix per preset via a local CMakeUserPresets.json and for troubleshooting.

3 Mathematical Model

A general overview of the equations that are solved by *CardioMechanics* is given in this section. This is a shortened version of the explanation given in [1].

3.1 Modeling Cardiac Mechanics

Under normal conditions, the heart undergoes large deformations during the contraction and relaxation phases of the cardiac cycle. The associated displacement of the myocardium can be found by relating the strain of the tissue with the stress generated by the contraction of myocytes together with the pressure that is exerted onto the endocardium by the blood inside the chambers.

Considering a continuous body $\Omega_0 \subset \mathbb{R}^3$ in its reference configuration, a deformation map $\varphi: (0, T] \times \Omega_0 \rightarrow \mathbb{R}^3$ can be defined that relates the reference coordinates $\mathbf{X} \in \Omega_0$ to the current coordinates $\mathbf{x} \in \Omega$ such that $\mathbf{x} = \varphi(\mathbf{X}, t)$ at time t . Next, the displacement field

of the body is defined by $\mathbf{d}(\mathbf{X}, t) = \varphi(\mathbf{X}, t) - \mathbf{X}$ together with the deformation gradient $\mathbf{F}(\mathbf{X}, t) = \nabla \varphi(\mathbf{X}, t) = \mathbf{1} - \nabla \mathbf{d}$. The deformation map is injective and orientation preserving, i.e. its Jacobian satisfies $J = \det \mathbf{F} > 0$.

The fundamental equation that describes the mechanical behavior and kinematics of a solid body is based on Newton's second law of motion. The so-called momentum equation:

$$\rho \frac{\partial^2 \mathbf{d}}{\partial t^2} - \nabla \cdot \boldsymbol{\sigma} - \rho \mathbf{b} = \mathbf{0}, \quad (1)$$

where ρ is the body's density, $\boldsymbol{\sigma}$ is the Cauchy stress, and $\mathbf{b}$ are body forces. Cauchy stress corresponds to the real or physical stress, since it refers to the current configuration. However, various stress measures can be delineated, exhibiting distinctions based on their respective frames of reference. The nominal stress tensor $\mathbf{P}$ is an asymmetric tensor defined as

$$\mathbf{P} = J \mathbf{F}^{-1} \boldsymbol{\sigma}, \quad (2)$$

and can be used to express the current force $d\mathbf{f}$ acting on a surface Γ in terms of the normal vector $\mathbf{n}_0$ and that surface in the reference configuration Γ_0 :

$$d\mathbf{f} = \mathbf{n}_0 \mathbf{P} d\Gamma_0. \quad (3)$$

However, due to its asymmetry, $\mathbf{P}$ is less attractive to work with. Cardiac tissue is considered an orthotropic material with fibers that have a certain orientation. Typically we are interested in the stress along the fibers. This can be done by using the second Piola-Kirchhoff stress tensor:

$$\mathbf{S} = J \mathbf{F}^{-1} \boldsymbol{\sigma} \mathbf{F}^{-T}. \quad (4)$$

Contrary to the nominal stress $\mathbf{P}$, $\mathbf{S}$ is symmetric and compared to the Cauchy stress does not change during rigid body rotations.

Passive material properties of the myocardium are assumed to be hyperelastic and quasi-incompressible. As a consequence, the second Piola-Kirchhoff stress $\mathbf{S}$ can be defined by a stored strain energy density function Ψ after introducing the right Cauchy-Green tensor $\mathbf{C} = \mathbf{F}^T \mathbf{F}$ and the Green-Lagrange strain tensor $\mathbf{E} = \frac{1}{2}(\mathbf{C} - \mathbf{1})$:

$$\mathbf{S} = 2 \frac{\partial \Psi(\mathbf{C})}{\partial \mathbf{C}} = \frac{\partial \Psi(\mathbf{E})}{\partial \mathbf{E}}. \quad (5)$$

Different constitutive laws have been introduced and used in the context of cardiac modeling. Due to the presence of fibers in cardiac tissue, these constitutive laws account for anisotropy by imposing a different elastic response along the directions of fibers $\mathbf{f}_0$, sheets $\mathbf{s}_0$, and sheet-normals $\mathbf{n}_0$.

To account for the active stress generated during cardiac contraction, $\mathbf{S}$ is additively decomposed into an active ($\mathbf{S}_a$) and a passive ($\mathbf{S}_p$) component such that

$$\begin{aligned} \mathbf{S}(\mathbf{d}, T_a(\mathbf{F})) &= \mathbf{S}_p + \mathbf{S}_a \\ &= 2 \frac{\partial \Psi(\mathbf{C})}{\partial \mathbf{C}} + T_a(\mathbf{F}) \frac{\mathbf{f}_0 \otimes \mathbf{f}_0}{\sqrt{\mathbf{F} \mathbf{f}_0 \cdot \mathbf{F} \mathbf{f}_0}}, \end{aligned} \quad (6)$$

where T_a is the contractile force. Active stress is modeled by upscaling the microscopic force T_a generated by cardiac muscle fibers to the tissue level. Thus, expressing it in terms of an internal stress that acts in the first principal direction of the tissue, namely the fibers $\mathbf{f}_0$.

3.2 Spatial Discretization

We use finite elements to transform Equation 1 into its discrete form. The notation $\dot{u} = \frac{\partial u}{\partial t}$ and $\ddot{u} = \frac{\partial^2 u}{\partial t^2}$ is adopted for time derivatives. Using index notation in terms of the displacement $\mathbf{d}$, Equation 1 is given by

$$\rho \ddot{d}_i - \frac{\partial}{\partial x_j} \sigma_{ji} - \rho b_i = 0 \quad (7)$$

The fundamental concept of the finite element method involves the approximation of d_i within the domain of interest Ω using a trial function $\tilde{d}_i$. This trial function comprises a defined finite set of appropriate basis functions N_i , referred to as element shape functions, which exhibit vanishing behavior on the displacement boundaries. The approximation of the displacement in component i at node I reads

$$d_i \approx \tilde{d}_i = d_{iI} N_I. \quad (8)$$

Since $\tilde{d}_i$ is an approximation to the solution, we are left with a residual R . Following Galerkin's Method of weighted residuals, we use an arbitrary weighting function $w_i = w_{iI} N_I$ (test function) from the same function space as the trial function and build the inner product with R :

$$\int_{\Omega} w_{iI} \left(\rho N_I N_J \ddot{d}_{jJ} - \frac{\partial N_I}{\partial x_j} \sigma_{ji} - \rho N_I b_i \right) d\Omega = 0. \quad (9)$$

This is the weak form of the linear momentum equation. Using the relations

$$\begin{aligned} J \sigma_{ji} &= F_{jk} P_{ki} = \frac{\partial x_j}{\partial X_k} P_{ki} \\ d\Omega &= J d\Omega_0 \\ \rho b_i &= \rho_0 b_i \Omega_0 \end{aligned}$$

we can transform the weak form into the reference domain Ω_0 (also known as total Lagrangian form):

$$\int_{\Omega_0} w_{iI} \left(\rho_0 N_I N_J \ddot{d}_{jJ} - \frac{\partial N_I}{\partial X_j} P_{ji} - \rho N_I b_i \right) d\Omega_0 = 0. \quad (10)$$

Finally, we use the derivative product formula in order to get rid of the derivative of the nominal stress and apply the Gauss's theorem on the result. Then we use that w_i vanishes on the traction boundary and obtain discrete equations of the weak form of the total Lagrangian formulation:

$$w_{iI} \int_{\Omega_0} \rho_0 N_I N_J \ddot{d}_{jJ} d\Omega_0 + w_{iI} f_{iI}^{\text{int}} - w_{iI} f_{iI}^{\text{ext}} = 0, \quad (11)$$

with the internal and external nodal forces

$$f_{iI}^{\text{int}} = \int_{\Omega_0} \frac{\partial N_I}{\partial X_j} P_{ji} d\Omega_0 \quad \text{and} \quad f_{iI}^{\text{ext}} = \int_{\Omega_0} N_I \rho_0 b_i d\Omega_0 + \int_{\Gamma_0} N_I t_i^0 d\Gamma_0, \quad (12)$$

respectively. Since the test functions w_{iI} are arbitrary, it follows

$$M_{ijIJ} \ddot{d}_{jJ} + f_{iI}^{\text{int}} - f_{iI}^{\text{ext}} = 0, \quad (13)$$

$$\text{where} \quad M_{ijIJ} = \delta_{ij} \int_{\Omega_0} \rho_0 N_I N_J d\Omega_0, \quad (14)$$

is the so-called mass matrix. Rewriting everything in matrix vector notation gives

$$\mathbf{M}\ddot{\mathbf{d}} + \mathbf{f}^{\text{int}}(\mathbf{d}(t), t) - \mathbf{f}^{\text{ext}}(\mathbf{d}(t), t) = \mathbf{0}. \quad (15)$$

If it is desired to take friction into account, the equation can be extended with a term for damping

$$\mathbf{M}\ddot{\mathbf{d}} + \mathbf{C}\dot{\mathbf{d}} + \mathbf{f}^{\text{int}}(\mathbf{d}(t), t) - \mathbf{f}^{\text{ext}}(\mathbf{d}(t), t) = \mathbf{0}. \quad (16)$$

The type of damping used in *CardioMechanics* is called Rayleigh damping with the damping matrix

$$\mathbf{C} = \alpha\mathbf{M} + \beta\mathbf{K} \quad (17)$$

using the linearization of the internal forces $\mathbf{f}^{\text{int}}(\mathbf{d}(t), t) \approx \mathbf{K} \cdot \mathbf{d}(t)$ and the stiffness matrix $\mathbf{K} := \nabla_{\mathbf{d}} \mathbf{f}^{\text{int}}(\mathbf{d}(t), t)$.

In the case of static equilibrium, where mass inertia can be neglected, all time derivatives vanish and Equation 16 reduces to

$$\mathbf{f}^{\text{int}}(\mathbf{d}) - \mathbf{f}^{\text{ext}}(\mathbf{d}) = \mathbf{0}. \quad (18)$$

3.3 Time Discretization

Now that the equations are discretized in space, the temporal derivatives $\dot{\mathbf{d}}$ and $\ddot{\mathbf{d}}$ have to be discretized in time as well. For better readability, we redefine $\mathbf{a} = \ddot{\mathbf{d}}$ and $\mathbf{v} = \dot{\mathbf{d}}$ and use $\tilde{()}$ for incremental solutions.

Since Equation 18 omits all time derivatives and yields a solution in static equilibrium with respect to the current boundary conditions, there is no need for time integration and the system

$$\mathbf{r}(\mathbf{d}) := \mathbf{f}^{\text{int}}(\mathbf{d}) - \mathbf{f}^{\text{ext}}(\mathbf{d}) = \mathbf{0}, \quad (19)$$

can be solved directly using Newton's method.

A suitable time integration method for Equation 16 is Newmark-beta time integration. It was designed for elasticity problems and can be set up as an explicit or an implicit method depending on the choice of parameters β and γ . Equation 16 is an initial value problem, which can be discretized using $\Delta t = t^{n+1} - t^n$, $\mathbf{d}(t^0) = \mathbf{d}^0$, $\mathbf{v}(t^0) = \mathbf{v}^0$, $\mathbf{a}(t^0) = \mathbf{a}^0$:

$$\mathbf{r}(\mathbf{d}^{n+1}) = \mathbf{M}\mathbf{a}^{n+1} + \mathbf{C}\mathbf{v}^{n+1} + \mathbf{f}^{\text{int}}(\mathbf{d}^{n+1}, t^{n+1}) - \mathbf{f}^{\text{ext}}(\mathbf{d}^{n+1}, t^{n+1}) \stackrel{!}{=} \mathbf{0} \quad (20)$$

To solve this equation with the Newmark-beta scheme, we first calculate predictor values for the displacement and the velocity which depend on the previous time step

$$\tilde{\mathbf{d}}^{n+1} = \mathbf{d}^n + \Delta t \mathbf{v}^n + \frac{\Delta t^2}{2} (1 - 2\beta) \mathbf{a}^n, \quad (21)$$

$$\tilde{\mathbf{v}}^{n+1} = \mathbf{v}^n + (1 - \gamma) \Delta t \mathbf{a}^n, \quad (22)$$

and substitute $\mathbf{d}^{n+1}$, $\mathbf{v}^{n+1}$, and $\mathbf{a}^{n+1}$ in Equation 20 with the following formulas

$$\mathbf{d}^{n+1} = \mathbf{d}^n + \Delta t \mathbf{v}^n + \frac{\Delta t^2}{2} ((1 - 2\beta) \mathbf{a}^n + 2\beta \mathbf{a}^{n+1}), \quad (23)$$

$$\mathbf{v}^{n+1} = \tilde{\mathbf{v}}^{n+1} + \gamma \Delta t \mathbf{a}^{n+1}, \quad (24)$$

$$\mathbf{a}^{n+1} = \frac{1}{\beta \Delta t^2} (\mathbf{d}^{n+1} - \tilde{\mathbf{d}}^{n+1}), \quad (25)$$

which leads to a system of nonlinear equations that only depends on $\mathbf{d}^{n+1}$:

$$\begin{aligned} \mathbf{r}(\mathbf{d}^{n+1}) = & \mathbf{M} \left[\frac{1}{\beta \Delta t^2} (\mathbf{d}^{n+1} - \tilde{\mathbf{d}}^{n+1}) \right] + \mathbf{C} \left[\frac{\gamma}{\beta \Delta t} (\mathbf{d}^{n+1} - \tilde{\mathbf{d}}^{n+1}) + \tilde{\mathbf{v}}^{n+1} \right] \\ & + \mathbf{f}^{\text{int}}(\mathbf{d}^{n+1}, t^{n+1}) - \mathbf{f}^{\text{ext}}(\mathbf{d}^{n+1}, t^{n+1}) = 0 \end{aligned} \quad (26)$$

The most robust method to solve non-linear algebraic equations such as Equation 26 is Newton's method. Linearizing Equation 26 using a Taylor expansion around the current value of the displacement $\mathbf{d}^{n+1}$ and dropping terms of higher order results in the linear model

$$\mathbf{r}(\mathbf{d}^{n+1}, t^{n+1}) + \mathbf{A} \Delta \mathbf{d}^{n+1} = 0, \quad (27)$$

with the system Jacobian matrix

$$\mathbf{A} = \frac{1}{\beta \Delta t^2} \mathbf{M} + \frac{\gamma}{\beta \Delta t} \mathbf{C} + \mathbf{K}, \quad (28)$$

where internal and external forces $\mathbf{K}$ are linearized in each Newton iteration:

$$\mathbf{K} \mathbf{d}^{n+1} \approx \mathbf{f}^{\text{int}}(\mathbf{d}^{n+1}, t^{n+1}) - \mathbf{f}^{\text{ext}}(\mathbf{d}^{n+1}, t^{n+1}). \quad (29)$$

Finally, we calculate the updated values for velocity and acceleration using Equation 24 and Equation 25.

3.4 The Inverse Problem: Active Stress Estimation

All of the above describes the *forward* problem: the active tension is prescribed by a tension model and the deformation follows. *CardioMechanics* can also solve the corresponding *inverse* problem: given a target deformed surface at every point in time, estimate the scalar active tension in each element that produces it. This is what the **ActiveStressEstimator** solver (Section 5.4) does. Note that the estimated quantity is one tension value per element, not a full stress tensor; it enters the constitutive response through the fiber direction exactly as an ordinary tension model would.

Let $\boldsymbol{\tau} \in \mathbb{R}^{n_e}$ collect the active tension of the n_e elements of interest, and let

$$\mathbf{r}(\mathbf{d}, \boldsymbol{\tau}) = \mathbf{f}^{\text{int}}(\mathbf{d}, \boldsymbol{\tau}) - \mathbf{f}^{\text{ext}}(\mathbf{d}) = \mathbf{0} \quad (30)$$

be the static equilibrium (18), which the estimator solves in the usual way for a given $\boldsymbol{\tau}$. The measurement is the signed gap $\mathbf{g} \in \mathbb{R}^m$ between the m tracked surface degrees of freedom (the *master* surface) and the target surface, measured along the master surface normal. It is supplied by the **ContactHandling** plugin, which is used here purely as a distance measurement and not to generate contact forces.

Sensitivity Differentiating (30) with respect to $\boldsymbol{\tau}$ at equilibrium gives

$$\mathbf{K} \frac{\partial \mathbf{d}}{\partial \boldsymbol{\tau}} + \mathbf{G} = \mathbf{0} \quad \text{with} \quad \mathbf{K} = \frac{\partial \mathbf{f}^{\text{int}}}{\partial \mathbf{d}} \quad \text{and} \quad \mathbf{G} = \frac{\partial \mathbf{f}^{\text{int}}}{\partial \boldsymbol{\tau}}, \quad (31)$$

so that the deformation responds to a change in active tension as $\partial \mathbf{d} / \partial \boldsymbol{\tau} = -\mathbf{K}^{-1} \mathbf{G}$. Restricting the rows to the master degrees of freedom yields the sensitivity matrix actually assembled by the solver,

$$\mathbf{A} = [\mathbf{K}^{-1}]_m \mathbf{G} \in \mathbb{R}^{m \times n_e}, \quad (32)$$

which is the negative of $\partial \mathbf{d} / \partial \boldsymbol{\tau}$ on those rows. The sign is carried in the update (38) below rather than in $\mathbf{A}$. Both $\mathbf{K}$ and $\mathbf{G}$ are obtained by finite differences of the nodal forces; $[\mathbf{K}^{-1}]_m$ is formed by a direct (MUMPS) factorization of $\mathbf{K}$ followed by one blocked solve against the unit loads of the master degrees of freedom.

Regularized Gauss-Newton step The estimation is ill-posed: $\mathbf{A}$ has at most rank m while $\boldsymbol{\tau}$ has n_e entries, and in practice $n_e \gg m$. It is therefore solved as a regularized linear least-squares problem in the increment $\Delta \boldsymbol{\tau}$, with the updated tension abbreviated as $\boldsymbol{\tau}^* = \boldsymbol{\tau} - \Delta \boldsymbol{\tau}$:

$$\begin{aligned} \Delta \boldsymbol{\tau} = \underset{\Delta \boldsymbol{\tau}}{\operatorname{argmin}} \Big\{ & \|\mathbf{A} \Delta \boldsymbol{\tau} - \mathbf{g}\|^2 + \lambda_1 \|\Delta \boldsymbol{\tau}\|^2 + \lambda_2 \|\boldsymbol{\tau}^*\|^2 \\ & + \frac{\lambda_3}{\Delta t} \|\boldsymbol{\tau}^* - \boldsymbol{\tau}^{n-1}\|^2 + \frac{\lambda_4}{\Delta t^2} \|\boldsymbol{\tau}^* - 2\boldsymbol{\tau}^{n-1} + \boldsymbol{\tau}^{n-2}\|^2 \\ & + \lambda_5 \|\mathbf{L} \Delta \boldsymbol{\tau}\|^2 + \lambda_6 \|\mathbf{L} \boldsymbol{\tau}^*\|^2 + \lambda_7 \|\mathbf{L} (\boldsymbol{\tau}^* - \boldsymbol{\tau}^{n-1})\|^2 \Big\}, \end{aligned} \quad (33)$$

where $\boldsymbol{\tau}^{n-1}$ and $\boldsymbol{\tau}^{n-2}$ are the converged tensions of the two preceding time steps. The individual terms are Tikhonov penalties with distinct purposes: λ_1 damps the step itself (a Levenberg-Marquardt term), λ_2 penalizes the magnitude of the tension, λ_3 and λ_4 penalize its first and second temporal differences, and λ_5 to λ_7 penalize spatial roughness of the step, of the tension, and of its temporal increment, respectively. The matrix $\mathbf{L}$ is the graph Laplacian of the element adjacency built from elements sharing a common face,

$$L_{ef} = \begin{cases} n_e & \text{if } e = f, \\ -1 & \text{if elements } e \text{ and } f \text{ share a face,} \\ 0 & \text{otherwise,} \end{cases} \quad (34)$$

with n_e the number of face neighbors of element e , so that $\|\mathbf{L} \boldsymbol{\tau}\|^2$ measures how much the tension of an element deviates from that of its neighbors.

Setting the derivative of (33) with respect to $\Delta \boldsymbol{\tau}$ to zero gives the normal equations that are actually assembled and factorized,

$$(\mathbf{A}^\top \mathbf{A} + \mathbf{R}) \Delta \boldsymbol{\tau} = \mathbf{A}^\top \mathbf{g} + \mathbf{c}, \quad (35)$$

with the regularization matrix $\mathbf{R}$ and the right hand side contribution $\mathbf{c}$

$$\mathbf{R} = \left(\lambda_1 + \lambda_2 + \frac{\lambda_3}{\Delta t} + \frac{\lambda_4}{\Delta t^2} \right) \mathbf{I} + (\lambda_5 + \lambda_6 + \lambda_7) \mathbf{L}^\top \mathbf{L}, \quad (36)$$

$$\begin{aligned} \mathbf{c} = & \lambda_2 \boldsymbol{\tau} + \frac{\lambda_3}{\Delta t} (\boldsymbol{\tau} - \boldsymbol{\tau}^{n-1}) + \frac{\lambda_4}{\Delta t^2} (\boldsymbol{\tau} - 2\boldsymbol{\tau}^{n-1} + \boldsymbol{\tau}^{n-2}) \\ & + \lambda_6 \mathbf{L}^\top \mathbf{L} \boldsymbol{\tau} + \lambda_7 \mathbf{L}^\top \mathbf{L} (\boldsymbol{\tau} - \boldsymbol{\tau}^{n-1}). \end{aligned} \quad (37)$$

This is a dense $n_e \times n_e$ system and is solved directly by an LU factorization. The tension is then updated and limited to a physiologically meaningful range,

$$\boldsymbol{\tau} \leftarrow \text{clamp}(\boldsymbol{\tau} - \Delta \boldsymbol{\tau}, 0, 400 \text{ kPa}), \quad (38)$$

where the increment itself is additionally limited to $|\Delta \tau_e| \leq 10 \text{ kPa}$ per element and iteration to keep the linearization valid.

Outer iteration Because (32) is only a linearization around the current state, each time step is iterated: the equilibrium (30) is solved, the gap $\mathbf{g}$ is measured, one step (35) to (38) is taken, and the equilibrium is re-solved with the updated tension. The iteration stops once $\|\mathbf{g}\|_2$ falls below `AbsTol`, or once it changes by less than `RelTol` between two iterations, and it is capped at five iterations per time step.

Note that the estimator inherits the linear system of the forward problem but adds a dense solve whose size scales with the number of elements, and that it is implemented for sequential execution only.

3.5 Units

In general, all units are given in the international system of units (SI). For the most important units you can refer to Table 1. If you are unsure about the unit of parameters not listed in Table 1, use base SI units without prefix.

Table 1: Units used in *CardioMechanics*.

Symbol	Input unit	Internal unit	Conversion	Comment
x, y, z	mm	m	10^{-3}	-
t	s	s	1	-
$\mathbf{d}$	m	m	1	-
ρ_0	kg m^{-3}	kg m^{-3}	1	-
T_a	Pa	Pa	1	-
$\mathbf{S}$	Pa	Pa	1	-
p	Pa	Pa; mmHg	1	decision by plugin

4 File Formats

4.1 Input Files

4.1.1 CardioMechanics Config File (*.xml)

This is the main configuration file that controls the mechanical simulations. It uses the extensible markup language (XML) to store and define all simulation parameters. The top level tags are defined as follows: **General**, **Mesh**, **Materials**, **Export**, **Solver**, and **Plugins**. All available settings inside these tags are described in detail within the [Simulation Framework](#).

```

1 <settings>A general description</settings>
2
3 <General>
4   <LogFile>   filename.txt   </LogFile>
5   <Verbose>   false/true     </Verbose>
6   <Debug>     false/true     </Debug>
7 </General>

```

```

8
9 <Mesh>
10 ...
11 </Mesh>
12
13 <Materials>
14 ...
15 </Materials>
16
17 <Export>
18 ...
19 </Export>
20
21 <Solver>
22 ...
23 </Solver>
24
25 <Plugins>
26 ...
27 </Plugins>

```

Listing 1: CardioMechanics Config File (*.xml)

4.1.2 Element file (*.ele)

The element file contains only the elements for the mechanical mesh and has the following structure:

```

1 <# elements> <Nodes per Element> <# Attributes>
2 <Element Index> <node 1> <node 2> ... <node n> <material index>
3 ...

```

Listing 2: Element file (*.ele)

Only P1 and P2 elements are supported, hence each element needs either $n = 4$ or $n = 10$ nodes.

4.1.3 Node File (*.node)

The node file contains only the vertices for the mesh and has the following structure:

```

1 <# nodes> <Dimension (must be 3)> <Boundary Conditions> (0: no
  boundary condition, 1: Dirichlet)> <BoundaryMarker (must be 0)>
2 <Node Index> <x> <y> <z> <Boundary Condition>
3 ...

```

Listing 3: Node File (*.node)

The boundary condition is set as an integer number between 0 and 7. These numbers translated to their respective binary representation describe the Dirichlet boundary condition in the axes z,y,x, i.e. 001 (dec: 1) -> x-direction fixed, 010 (dec: 2) -> y-direction fixed, 100 (dec: 4) -> z-direction fixed, 111 (dec: 1+2+4 = 7) all directions fixed. Combinations of the aforementioned fixations are possible as well, i.e. you can fix a node in the x,y-plane by using the integer number 3 (binary: 011) etc.

4.1.4 Surface Elements File (*.sur)

This file contains a list of all surface triangle node indices of the mechanical mesh and its attributes.

```
1  <# elements> <Nodes per Element> <# Attributes (min 2; max 3)>
2  <surface element Index> <node 1> <node 2> ... <node n> <material
   index> <surface index> <surface traction scaling>
3  ...
```

Listing 4: Surface Elements File (*.sur)

Only P1 and P2 elements are supported, hence $n = 3$ or $n = 6$. You need a minimum of two or a maximum of three attributes defined for each surface element. Attribute 1 is the material index, which is used as a material tag during export. Attribute 2 is the surface index, which is used by boundary conditions. Attribute 3 is optional and sets a scaling factor which can be applied to epicardial boundary conditions ([ContactHandling](#), [RobinBoundary](#), [RobinBoundaryGeneral](#)).

4.1.5 Fiber Orientation File (*.bases)

This file contains a list of all fiber orientations of the mechanical mesh. If you intend to use P2 elements, you have to submit the fiber orientations for each quadrature point as well. Either way, the first orientation is always the centroid of the element. This means it is possible to use a file for P2 elements with P1 elements as well, since the program will only read the number of quadrature points given in the file.

```
1  <# elements> <# qp>
2  <element index> <fiber_x> <fiber_y> <fiber_z> <sheet_x> <sheet_y>
   <sheet_z> <normal_x> <normal_y> <normal_z>
3  ...
```

Listing 5: Fiber Orientation File (*.bases)

4.1.6 Local Activation Time File (*.txt)

This file contains activation times for each element of the mesh that defines the start of the contraction cycle in the tension model. If a tension model requires a calcium transient, you need to make sure to provide one. This is only relevant to the [paragraph 5.2.2](#) model, which has multiple calcium transients to choose from already implemented. Leave an empty line at the end of the file.

```
1  <# elements> 1
2  <element 1> <activation time in s>
3  <element 2> <activation time in s>
4  <element 3> <activation time in s>
5  ...
6  <element n> <activation time in s>
```

Listing 6: Local Activation Time File (*.txt)

4.1.7 Target Surface Files (*.list, *.dat)

These files provide time-dependent nodal coordinates to the [PointsCtrl](#) plugin and thereby the target surfaces of the inverse problem (Section 5.4). They are usually written by the [CreateTargetSurfaces](#) script rather than by hand.

The list file is a plain text table that assigns one binary data file to each point in time. Paths are interpreted relative to the directory of the list file, empty lines are skipped, and lines whose first non-blank character is # are treated as comments.

```
1 <time in s> <path to data file>
2 <time in s> <path to data file>
3 ...
```

Listing 7: Target surface list file (*.list)

Each referenced data file is binary and contains the coordinates of all controlled nodes at that point in time, in the order in which the nodes are given to the plugin. The leading count is the number of `double` values that follow, i.e. three times the number of nodes. Values are written in the native byte order of the machine.

```
1 int32 3 * <# nodes>
2 double x[0]
3 double y[0]
4 double z[0]
5 double x[1]
6 ...
```

Listing 8: Target surface data file (*.dat)

The same binary layout is used wherever *CardioMechanics* reads tabulated data over time from a file, for instance by the [File](#) tension model (Section 5.2.2), which stores one tension value per element instead of three coordinates per node.

4.2 Output Files

4.2.1 VTK Unstructured Grid Files (*.vtu)

This file contains all data specified by the [Export](#) options and the `SolverPlugin::Export()` function of each active plugin as long as the plugin supports exporting to VTK. Refer to [Table 3](#) for an overview of export options to the VTK file type. A file is written at each `Export.TimeStep`.

4.2.2 ParaView Data File (*.pvd)

This file can be used to open, view, and post process the time series of [VTK Unstructured Grid Files \(*.vtu\)](#) files in [ParaView](#). Refer to the [ParaView Wiki on data formats](#) for more information.

4.3 acCELLerate Specific Input Files

The input files listed in the following sections are specific to *acCELLerate* and are only required if you intend to use the solver plugin [acCELLerate](#) to run electromechanically

coupled simulations.

4.3.1 Config File (*.act)

This is the main configuration file for EP tissue simulations. Not all listed entries are needed for, e.g. Monodomain simulations, but possible entries are

1	Resprefix	The prefix of the result data
2	Condition	Name of the condition file
3	Sensor	Name of the sensor file
4	CalcLength	Calculation length [s]
5	DTCell	Time step of cell model calculation [s]
6	DTExtra	Time step of extracellular calculation [s]
7	DTIntra	Time step of intracellular calculation [s]
8	BeginSave	Time at which saving of results starts [s]
9	DTSave	Time step of saving [s]
10	CellModelFile	Name of the cell model file
11	Material	Name of vector describing tissue classes
12	MaterialFibro	Name of vector describing fibroblast classes
13	MatrixIntra	Name of intracellular conductivity matrix
14	MatrixExtra	Name of extracellular conductivity matrix
15	MatrixExtraCombined	Name of extracellular conductivity matrix that is already combined with Intra (and Fibro)
16	MatrixFibro	Name of fibroblast conductivity matrix
17	BetaMyoFib	Number of Myo-Fibro gap junctions [1/m ³]
18	RMyoFib	Resistor of single myo-fibro gap junction [Ohm]
19	VolumeMyo	Relative amount of myocyte volume
20	VolumeExtra	Relative amount of extracellular volume
21	VolumeFibro	Relative amount of fibroblast volume
22	Domain	Type, either "Mono", "Bi", or "Tri"
23	Verbose	Print verbose information into terminal
24	Protocol	Name of verbose information file
25	HeteroFileIntra	Name of the heterogeneous cell model file (entries: <vector file name> <parameter name>)
26	HeteroFileFibro	Name of the heterogeneous fibro model file (entries: <vector file name> <parameter name>)
27	LoadBackup	Loads backup from standard file of <file> if declared
28	SaveBackup	Saves backup to standard file of <file> if declared
29	DTBackup	Time step of backup
30	Results	List of variables to be saved (e.g., Vm, Ve, AT, m, Na _i , I _{Na} , and any variable of the cell model)
31	Implicit	Number of Jacobi iterations for implicit calculation (!Only for Monodomain)
32	Force	Also calculate and output force
33	MassMatrixIntra	Name of intracellular mass matrix
34	ThetaIntra	Theta for intra PDE solver scheme. [0.0..1.0] 0=explicit, 1=implicit (default: 0 (FD) or 0.5 (FE))
35	MembraneCapacitance	Membrane capacitance per unit area, number or vector (F/m ²)
36	SurfaceToVolume	Myocyte surface to volume ratio, number or vector (1/m, default: cell model)
37	Gauss	Matrices for gauss interpolation and integration
38	ActivationThreshold	TMV threshold for activation time (default: 0V).
39	CurrentScheme	Godunow, SVI, ICI, None. SVI and None require Gauss matrices.
40	IntraIndexSet	PETSc IndexSet of the intracellular domain (if smaller than the xtracellular domain)

```

41 Compress      Compress output vectors using deflate/gzip.
42 NoExport      Deactivate Export

```

Listing 9: acCELLerate configuration file (*.act)

4.3.2 Condition File (*.acnd)

This file contains the stimuli and initial/boundary conditions for an EP simulation.

```

1  <node index> <amplitude> <cycle length> <duration> <temporal offset>
   Ii | Ie | If | Vm | Ve | Vf
2  ...

```

Listing 10: acCELLerate condition file (*.acnd)

The type of stimulus (last column) is given as a combination of first letter: either I(current) or V(voltage); domain of stimulus as second letter: intracellular=i, extracellular=e or fibroblastic=f.

4.3.3 Cell Model File (*.cmf)

Within the cell model file, the tissue classes from the geometry file are assigned to cell model types and force models.

```

1  <first tissue class> <elphpy model> <force model>
2  <second tissue class> <elphpy model> <force model>
3  <third tissue class> <elphpy model> <force model>
4  ...

```

Listing 11: acCELLerate cell model file (*.cmf)

Elphy model and force model are given as the path to the .ev-file and .fv-file you want to use.

4.3.4 Material File (*.def)

Defines material properties, such as conductivities. Each material that exists in the *Material* array of the geometry has to be defined in this file with the following entries:

```

1 [NAME MATERIALNUMBER COLORCODE]
2 Kappa          FLOAT FLOAT
3 AnisotropyX    FLOAT
4 AnisotropyY    FLOAT
5 AnisotropyZ    FLOAT

```

Listing 12: acCELLerate material file (*.def)

Kappa is the conductivity (second column, in S/m) at a given frequency (first column). The COLORCODE value is currently not in use and can be set to an arbitrary number. MATERIALNUMBER should be of type INT and should stay in the range 1 to 255 (same in the geometry array!). You can use the anisotropy parameters to define different conductivities in fiber (AnisotropyX), sheet (AnisotropyY), and normal (AnisotropyZ) directions. Kappa by itself defines the transversal conductivities, such that only AnisotropyX would need to be defined in case of transverse isotropy. A second option to define a material is by using

a definition of another material by referencing it with a hashtag followed by the material number if the conductivities are the same. In the following example, we set a general material for the ventricles and atria to define the left/right atrium and ventricle.

```

1 [Ventricle_myocardium 100 0]
2 Kappa 0 0.28
3 AnisotropyX 1.0
4 AnisotropyY 0.65
5 AnisotropyZ 0.35
6
7 [Atria_myocardium 200 0]
8 Kappa 0 0.1823
9 AnisotropyX 3.75
10
11 [Ventricle_rechts 2 0]
12 #100
13
14 [Ventricle_links 3 0]
15 #100
16
17 [right_atrium 32 0]
18 #200
19
20 [left_atrium 33 0]
21 #200

```

Listing 13: acCELLerate material file example

4.3.5 Sensor File (*.txt)

The sensor files contain the information about the sensors that are placed in a simulated tissue. A sensor records any output parameter of the cell model in the applied node and saves the values in a textfile.

```

1 <node index> <file name> <offset> <t_increment> Ii|Ie|If|Vm|Ve|Vf (or
  any output parameter of the applied cellmodel)
2 ...

```

Listing 14: acCELLerate sensor file (*.txt)

4.3.6 CellModel Parameter File (*.ev)

Cellmodel files define parameters and initial values for ionic models. The general structure is given below. Parameter files for all ionic models can be found in [electrophysiology/data/](#).

```

1 CELLMODEL*IBT*KA
2 CELLMODELNAME
3 # of parameters to read
4 ...
5 ...
6 ...

```

Listing 15: Ionic model parameter file (*.ev)

4.3.7 ForceModel Parameter File (*.fv)

Files with *.fv* ending contain parameters and initial values for tension models. Their structure is the same as for *.ev* files described above.

5 Simulation Framework

CardioMechanics is a robust object-oriented programming framework featuring meticulously crafted interfaces to facilitate extensible implementations. This framework seamlessly couples various model components while ensuring swift solutions through efficient utilization of computational resources via software parallelization using MPI and adaptive time increments. In this section, we delve into the pivotal elements of *CardioMechanics*, the framework tailored for conducting cardiac biomechanics (CB) simulations. This framework is intricately connected to two external libraries: the VTK library, employed for efficient data export, and the PETSc library, used for matrix vector operations and to solve the dynamic linear and non-linear equation systems. Programmed in C++, *CardioMechanics* adopts a comprehensive object-oriented programming structure, emphasizing its role as a multiscale electromechanical simulation framework. It uses the method of finite elements, to solve the equilibrium of forces within the simulation domain. The fundamental class structure of *CardioMechanics* is shown in [Figure 1](#) and consists of the following components:

CBCardioMechanics presents a user-friendly command line interface, proficient in parsing user input and initiating core components based on the provided instructions. Configuration files, adhering to the XML format, serve as conduits for parameters that configure the Model/Mesh, Solver, Exporter, Materials, and all SolverPlugins. Notably, all parameters are subject to override via the command line, facilitating swift testing of various configurations, such as adjusting time steps. Additionally, users have the flexibility to select different levels of verbosity for logging data.

CBModel manages all data encompassing geometric information, undergoing population during mesh loading courtesy of CBModelLoader. Its primary role lies in conveying geometric details to the solver. Additionally, it plays a pivotal role in exporting simulation data to [VTK Unstructured Grid Files \(*.vtu\)](#) files. Consequently, it undergoes updates with the most recent node positions and is enriched with supplementary information, including active stress, relative fiber length, and the Cauchy deformation tensor. This augmented dataset is then handed over to CBModelExporter.

CBSolver tackles a nonlinear problem within each time step, leveraging PETSc's SNES-Solve function to address nonlinear equation systems. It incorporates time integration in one of its specialized derived classes, namely CBSolverStatic and CBSolverNewmarkBeta. Responsible for overarching tasks such as step size computation, invocation of solver plugins, and mesh exporting, the CBSolver's Run() function orchestrates these operations. Featuring distinct data structures formatted for efficient computation, including parallel PETSc matrices and vectors, CBSolver necessitates conversion to a CBModel object before exporting. Instead of directly computing new node coordinates, the SolverStep() function

calculates displacements in each time step, subsequently adding them to node positions from the preceding time step.

CBSolverPlugin serves as a comprehensive interface to extend the solver’s capabilities. The class defines essential functions such as `Init()`, `Apply()`, `StepBack()`, and `Export()`. These functions are invoked before commencing the simulation, during each solver step for updating internal information, after each unsuccessful solver step for reverting to the previous state, and after each successful solver step, complying with an export time condition, for writing information into the model and/or a text file. SolverPlugins wield influence over the subsequent time step through return codes like `CBStatus::SUCCESS` (indicating permission for the solver to advance to the next time step), `CBStatus::FAILED` (suggesting a reduction in time step due to plugin dissatisfaction with results), or `CBStatus::REPEAT` (prompting the solver to recompute the current time step, enabling plugins to iterate multiple times).

CBElement The element structure comprises two distinct types: `CBElementSolid` and `CBElementSurface`. `CBElementSolid` is equipped with functions that populate the stiffness matrix with node derivative entries in a PETSc-compatible format. These functions are implicitly invoked by PETSc’s `SNESolve()` function, which employs the solver’s `NodalForcesHelperFunction()` to call each solid element’s `CalcNodalForces()`. This calculation determines the element’s contribution to the system matrix, necessitated by the intricate structure dictated by PETSc’s `SNESolve()`. Unfortunately, this core functionality of the solver had to be implemented in a somewhat convoluted manner due to these constraints. Each `CBElementSolid` is self-aware, maintaining knowledge of its current and initial node positions, fiber directions, material model, and tension model. This information is instrumental in computing its deformation tensor and determining the resulting contribution to the global energy derivatives. Each solid element is associated with a `CBConstitutiveModel` for passive stress computation and a `CBTensionModel` for active stress computation during its `CalcNodalForces()` operation. In contrast, `CBElementSurface` is designed to handle surface forces application, volume computation, and identification of contact partners for the contact handling algorithm.

CBElementAdapter serves as a central hub, holding pointers to critical global and element-specific components within *CardioMechanics* for the sake of efficiency. Examples include pointers to the solver, constitutive model, time-stepping object, and other essential elements. The primary purpose is twofold: firstly, to grant each element read access to information that is technically not available within the element, such as global variables. To minimize memory consumption, node positions are stored only once in memory, facilitated by the solver’s solution vector, with `CBElementAdapter` providing the location in memory. Secondly, this global object enables elements to directly contribute to mass and stiffness matrices within the solver’s data objects, fostering seamless integration of element-specific computations. This approach optimizes both memory usage and computational efficiency throughout *CardioMechanics*.

CBConstitutiveModel provides the functions `CalcEnergy()` and `CalcPK2Stress()`, empowering each solid element to compute its deformation energy based on a provided deformation tensor, considering the passive material properties.

CBTensionModel offers functions for computing active energy contributions arising from contracting muscle fibers, utilizing a given deformation tensor. Specifically, `CalcActiveTension()` calculates the tension in the fiber direction, while `CalcActiveStressTensor()` determines the contribution to the PK2 stress. The structure closely aligns with `CBConstitutiveModel`, reflecting a consistent approach to handling active stress computations within the framework.

The object-oriented nature of the codebase allows for the seamless interchangeability and extension of specialized modules. For example, specializations of the `CBElement` class include `CBElementSolidT4` and `CBElementSolidT10`. Similarly, specializations of the `CBConstitutiveModel` class encompass `CBConstitutiveModelGuccione` and `CBConstitutiveModelMooneyRivlin`. Despite their unique functionalities, all these specializations adhere to the same interface functions, ensuring compatibility and ease of integration within the system.

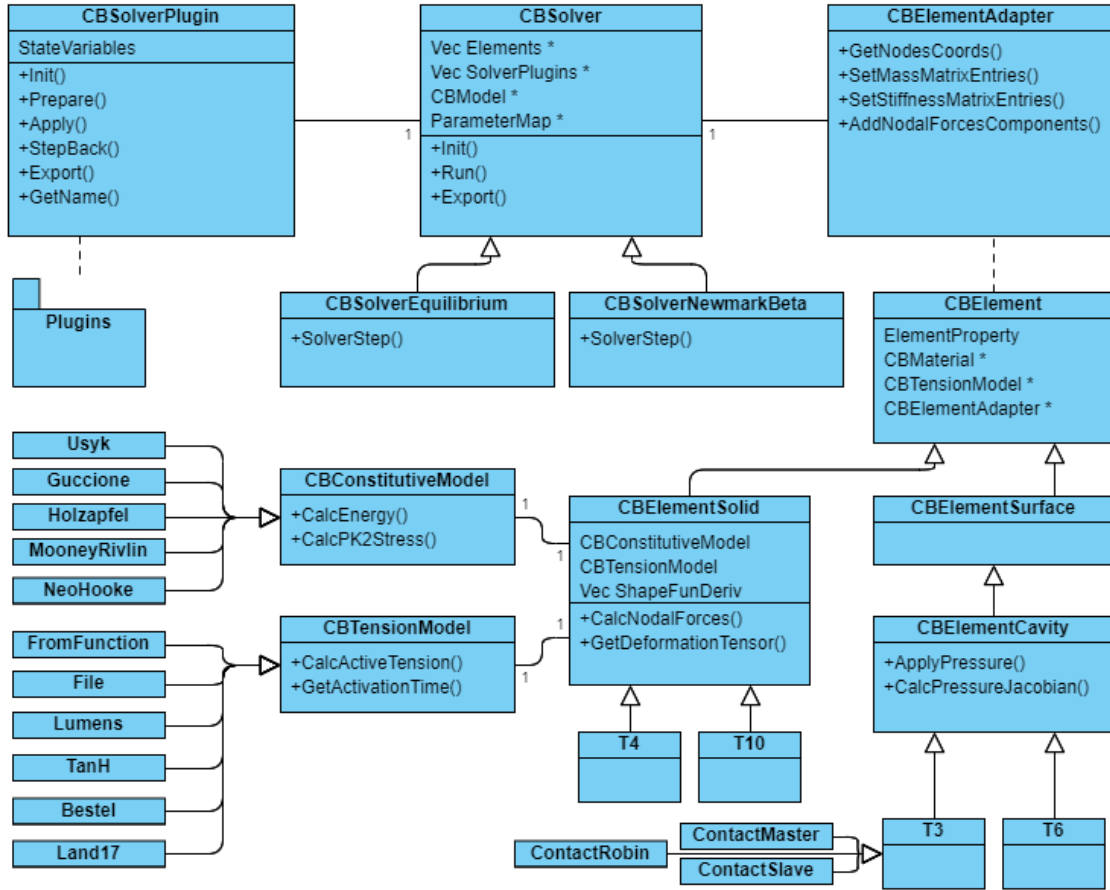


Figure 1: Fundamental structure of the simulation framework *CardioMechanics*.

5.1 Mesh

The tag `Mesh`, contains all parameters for the configuration of the model/geometry. The mesh has to be provided in the TetGen format. See [Node File \(*.node\)](#), [Element file \(*.ele\)](#), [Surface Elements File \(*.sur\)](#), and [Fiber Orientation File \(*.bases\)](#) for details about this format. Only linear (T4) and quadratic tetrahedral (T10) elements are supported as volume elements. During the initialization process, you can choose the method of domain decomposition. This can either be done by node index (`None`) or by principal component analysis (PCA) of initial node coordinates. A third variant is available using the `Solver.DomainDecomposition` tag, which uses information from the stiffness matrix to minimize communication cost (becomes more relevant in HPC applications).

Surfaces and surface types for boundary condition applications have to be defined using the `Surface` tag. Only linear (T3) and quadratic (T6) triangular elements are supported. However, most plugins supplying traction forces are only implemented for linear triangles. See the code example below for available options.

The tag `Transform` can be used to transform linear to quadratic elements on the fly. This is only recommended for testing purposes.

Finally, a [Local Activation Time File \(*.txt\)](#) can be defined, if you want to run mechanical simulations with precalculated electrophysiological activations, e.g. from solutions to the eikonal equation.

```
1 <Mesh>
2   <Type>      T4/T10      </Type>
3   <Format>    Tetgen      </Format>
4   <Sorting>   None/PCA    </Sorting>
5   <Tetgen>
6     <Unit>    1e-3        </Unit>
7     <Nodes>   filename.node </Nodes>
8     <Elements> filename.ele </Elements>
9     <Surfaces> filename.sur </Surfaces>
10    <Bases>   filename.bases </Bases>
11    <DetermineNeighbors> true/false </DetermineNeighbors>
12    <EnforceOrthonormalBases> true/false </EnforceOrthonormalBases>
13  </Tetgen>
14
15  <!-- Define a surface with material number XXX and one of the
16  following surface types:
17  CAVITY - for closed surfaces used in the circulatory system
18  T3, T6 - arbitrary surfaces
19  CONTACT_ROBIN - for a generalized Robin boundary condition
20  CONTACT_MASTER/CONTACT_SLAVE - for the contact handling plugin based
21  on Fritz et al. 2013 -->
22  <Surfaces>
23    <Surface_XXX> surfaceType </Surface_XXX>
24  </Surfaces>
25
26  <!-- Transform linear elements to quadratic elements on the fly -->
27  <Transform>
28    <T4toT10>   false </T4toT10>
29    <T3toT6>    false </T3toT6>
30  </Transform>
31
32  <!-- Define a filename with local activation times (LAT) -->
33  <LAT>
```



```

32     <FilePath> filename.txt </FilePath>
33     </LAT>
34 </Mesh>

```

Listing 16: .xml settings for mesh initialization

5.2 Materials

The tag **Materials** contains all parameters for the definition of material properties. Using the tag **Global.Damping**, the user should define the parameters for Rayleigh damping of the form $\mathbf{C} = \alpha\mathbf{M} + \beta\mathbf{K}$. Furthermore, the user can set default material parameters using the tag **Mat_Default**. The parameters defined under this scope are valid for all materials unless specific material definitions are available. You can define different constitutive models and tension models for each individual material if you wish to do so.

In the example below, default parameters for the density and the Usyk constitutive law are set. If the tag **IgnoreCorruptElements** is set to **true**, elements with $\det(\mathbf{F}) \leq 0$ are ignored and do not contribute to the stiffness matrix. For obvious reasons, this setting is not recommended for use. Additionally, a material defined by number 30 is defined. It uses the Usyk constitutive law with additional scaling applied to the default parameters as well as the Land17 tension model. **TensionMax** is set to 1000, since tension in the Land model is in kPa and we need to scale it to Pa.

In the following sections, the configurations for all available constitutive laws and tension models are discussed.

```

1 <Materials>
2   <Global>
3     <Damping>
4       <Rayleigh>
5         <Alpha> 500 </Alpha>
6         <Beta> 0.005 </Beta>
7       </Rayleigh>
8     </Damping>
9   </Global>
10
11   <Mat_Default>
12     <IgnoreCorruptElements> true/false </IgnoreCorruptElements>
13     <Density> 1082 </Density>
14     <Usyk>
15       <a>880</a> <!-- Pa -->
16       <bff>8</bff>
17       <bss>6</bss>
18       <bnn>3</bnn>
19       <bfs>12</bfs>
20       <bfn>3</bfn>
21       <bns>3</bns>
22       <k>650e3</k> <!-- Pa -->
23     </Usyk>
24   </Mat_Default>
25
26   <Mat_30>
27     <Type>Usyk</Type>
28     <Usyk>
29       <aScale>0.07</aScale>
30       <bScale>3.55</bScale>

```

```

31     </Usyk>
32
33     <TensionMax>1e3</TensionMax>
34     <TensionModel>Land17</TensionModel>
35     <Land17>
36         <CalciumTransientType>Elphy</CalciumTransientType>
37         <rateDependancy>0N</rateDependancy>
38         <cycleLength>0.8</cycleLength>
39         <beta0>0.7</beta0>
40         <beta1>-1.2</beta1>
41         <Ca50>0.805</Ca50>
42         <Tref>440</Tref>
43     </Land17>
44 </Mat_30>
45
46 ...
47 </Materials>

```

Listing 17: .xml settings for material definitions

5.2.1 Constitutive Models

The material constitutive laws that are used in the balance of momentum equation relate the strain of the body with its stress. However, in cardiac mechanics, the focus is on elastic materials or in particular hyperelastic ones, i.e. the behaviour of the material is path-independent. As a consequence, we can define a stored strain energy density function Ψ to express the first Piola-Kirchhoff stress tensor as

$$\mathbf{P}(\mathbf{F}(\mathbf{X}), \mathbf{X}) = \frac{\partial \Psi(\mathbf{F}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{F}}, \quad (39)$$

or in terms of the symmetric second Piola-Kirchhoff stress tensor

$$\mathbf{S}(\mathbf{C}(\mathbf{X}), \mathbf{X}) = 2 \frac{\partial \Psi(\mathbf{C}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{C}} = \frac{\partial \Psi(\mathbf{E}(\mathbf{X}), \mathbf{X})}{\partial \mathbf{E}}. \quad (40)$$

If the relationship between Ψ and $\mathbf{C}$ is independent of the material axes, Ψ can be expressed as a function of the invariants of $\mathbf{C}$ as

$$\Psi(\mathbf{C}(\mathbf{X}), \mathbf{X}) = \Psi(\text{I}_{\mathbf{C}}, \text{II}_{\mathbf{C}}, \text{III}_{\mathbf{C}}, \mathbf{X}), \quad (41)$$

where the invariants are defined as

$$\text{I}_{\mathbf{C}} = \text{tr } \mathbf{C} = \mathbf{C} : \mathbf{1} \quad (42)$$

$$\text{II}_{\mathbf{C}} = \frac{1}{2} ((\text{tr } \mathbf{C})^2 - \text{tr } \mathbf{C}^2) \quad (43)$$

$$\text{III}_{\mathbf{C}} = \det \mathbf{C} = J^2. \quad (44)$$

It can also be convenient to consider isochoric invariants, replacing $\mathbf{C}$ with $\bar{\mathbf{C}} = J^{-2/3} \mathbf{C} = \text{III}_{\mathbf{C}}^{-1/3} \mathbf{C}$. Furthermore, pseudo invariants can be introduced that describe the squared stretch along microstructural directions in anisotropic materials such as the myocardium:

$$\text{I}_{ab} = \mathbf{C} : \text{sym}(\mathbf{e}_a \otimes \mathbf{e}_b), \quad a, b \in \{f, s, n\}, \quad (45)$$

where $\text{sym}(\mathbf{A}) = \frac{1}{2}(\mathbf{A} + \mathbf{A}^T)$ is the symmetric transformation. Near incompressibility can be achieved in hyperelastic materials by adding a volumetric energy component $U(J)$ to the distortional component $\Psi(\bar{\mathbf{C}})$:

$$\Psi(\mathbf{C}) = \Psi(\bar{\mathbf{C}}) + U(J), \quad (46)$$

and

$$\mathbf{S} = 2 \frac{\partial \Psi(\bar{\mathbf{C}})}{\partial \mathbf{C}} + 2 \frac{dU(J)}{dJ} \frac{\partial J}{\partial \mathbf{C}}. \quad (47)$$

In the following, the constitutive models that are implemented in *CardioMechanics* are described with their respective parameters.

Guccione The incompressible strain energy density by Guccione et al. [2] is given by

$$\begin{aligned} \Psi^{(G)}(\mathbf{C}) &= \frac{\kappa}{2}(J-1)^2 + \frac{\mu}{2}(\exp(Q) - 1), \\ Q &= b_f E_{ff}^2 + b_t (E_{ss}^2 + E_{nn}^2 + E_{ns}^2 + E_{sn}^2) + b_{fs} (E_{fs}^2 + E_{sf}^2 + E_{fn}^2 + E_{nf}^2), \end{aligned} \quad (48)$$

where $E_{ij} = \mathbf{E} \mathbf{i}_0 \cdot \mathbf{j}_0$ for $i, j \in \{f, s, n\}$ are the entries of the Green-Lagrange strain tensor $\mathbf{E} = \frac{1}{2}(\mathbf{C} - \mathbf{1})$. The 2nd Piola-Kirchhoff stress is given by

$$\mathbf{S} = \mu \exp(Q) (b \odot E)_{ij} + \kappa (J-1) J \mathbf{C}^{-1} \quad \text{with} \quad \mathbf{b} = \begin{pmatrix} b_f & b_{fs} & b_{fs} \\ b_{fs} & b_t & b_t \\ b_{fs} & b_t & b_t \end{pmatrix}, \quad (49)$$

where $(A \odot B)_{ij} = (A)_{ij}(B)_{ij}$ is the Hadamard or Schur product of the matrices $\mathbf{A}$ and $\mathbf{B}$.

```

1      <Type>Guccione</Type>
2      <Guccione>
3          <C>FLOAT</C> <!-- shear modulus [Pa] -->
4          <bf>FLOAT</bf>
5          <bt>FLOAT</bt>
6          <bfs>FLOAT</bfs>
7          <K>FLOAT</K> <!-- bulk modulus [Pa] -->
8          <aScale>FLOAT</aScale> <!-- optional scaling for C (default 1) -->
9          <bScale>FLOAT</bScale> <!-- optional scaling for b (default 1) -->
10     </Guccione>

```

Listing 18: .xml settings for Guccione material

MooneyRivlin The Mooney-Rivlin material is a commonly used isotropic material law. The following more general form is implemented in *CardioMechanics*:

$$\Psi^{(MR)}(\mathbf{I}_C, \mathbf{II}_C) = \sum_{i=0}^2 \sum_{j=0}^2 \bar{c}_{ij} (\mathbf{I}_C - 3)^i (\mathbf{II}_C - 3)^j \quad \text{with} \quad \bar{c}_{00} = 0, \quad (50)$$

using a penalty function expressed as $\ln(\mathbf{III}_C) = 0$ and the modified strain energy potential

$$\hat{\Psi} = \Psi^{(MR)} - (c_{10} + 2c_{01}) \ln(\mathbf{III}_C) + \frac{1}{2} \kappa (\ln(\mathbf{III}_C))^2. \quad (51)$$

The 2nd Piola-Kirchhoff stress tensor is given by

$$\mathbf{S} = 2 \frac{\partial \hat{\Psi}}{\partial \mathbf{C}} = 2 \frac{\partial \Psi^{(MR)}}{\partial \mathbf{C}} + 2(- (c_{10} + 2c_{01}) + \kappa \ln(\mathbf{III}_C)) \mathbf{C}^{-1}, \quad (52)$$

where

$$2 \frac{\partial}{\partial \mathbf{C}} \Psi^{(\text{MR})} = 2 \left(\frac{\partial \Psi^{(\text{MR})}}{\partial \mathbf{I}_{\mathbf{C}}} \frac{\partial \mathbf{I}_{\mathbf{C}}}{\partial \mathbf{C}} + \frac{\partial \Psi^{(\text{MR})}}{\partial \Pi_{\mathbf{C}}} \frac{\partial \Pi_{\mathbf{C}}}{\partial \mathbf{C}} \right) \quad (53)$$

$$= 2 \left(\frac{\partial \Psi^{(\text{MR})}}{\partial \mathbf{I}_{\mathbf{C}}} + \mathbf{I}_{\mathbf{C}} \frac{\partial \Psi^{(\text{MR})}}{\partial \Pi_{\mathbf{C}}} \right) \mathbf{1} - 2 \frac{\partial \Psi^{(\text{MR})}}{\partial \Pi_{\mathbf{C}}} \mathbf{C}, \quad (54)$$

with

$$\frac{\partial \Psi^{(\text{MR})}}{\partial \mathbf{I}_{\mathbf{C}}} = c_{10} + 2c_{20}(\mathbf{I}_{\mathbf{C}} - 3) + c_{11}(\Pi_{\mathbf{C}} - 3), \quad (55)$$

$$\frac{\partial \Psi^{(\text{MR})}}{\partial \Pi_{\mathbf{C}}} = c_{01} + 2c_{02}(\Pi_{\mathbf{C}} - 3) + c_{11}(\mathbf{I}_{\mathbf{C}} - 3). \quad (56)$$

```

1 <Type>MooneyRivlin</Type>
2 <MooneyRivlin>
3   <C10>FLOAT</C10> <!-- [Pa] -->
4   <C01>FLOAT</C01> <!-- [Pa] -->
5   <C20>FLOAT</C20> <!-- default 0 -->
6   <C02>FLOAT</C02> <!-- default 0 -->
7   <C11>FLOAT</C11> <!-- default 0 -->
8   <B>FLOAT</B> <!-- bulk modulus [Pa] -->
9 </MooneyRivlin>

```

Listing 19: .xml settings for general Rivlin material

Holzapfel The incompressible Holzapfel-Ogden strain energy density [3] can be written as a sum of isotropic and anisotropic components

$$\begin{aligned} \Psi^{(\text{HO})}(\mathbf{C}) = & \frac{a}{2b} [\exp(b(\bar{\mathbf{I}}_{\mathbf{C}} - 3)) - 1] + \sum_{kl \in S} \frac{a_{kl}}{2b_{kl}} [\exp(b_{kl}(\mathbf{I}_{kl} - \delta_{kl})^2) - 1] \\ & + \frac{\kappa}{4} (J^2 - 1 - 2 \ln J), \end{aligned} \quad (57)$$

where δ_{kl} denotes the Kronecker delta (zero unless $k = l$, in which case it is 1) and $S = \{ff, ss, fs\}$. The anisotropic invariants $\mathbf{I}_{ff}$ and $\mathbf{I}_{ss}$ do not support compression. Therefore, the associated strain energy density terms are zero if $\mathbf{I}_{ff}, \mathbf{I}_{ss} < 1$. The 2nd Piola-Kirchhoff stress tensor is given by

$$\begin{aligned} \mathbf{S} = & \frac{\kappa}{2} \left(J - \frac{1}{J} \right) J \mathbf{C}^{-1} + \frac{a}{J^{2/3}} \exp(b(\bar{\mathbf{I}}_{\mathbf{C}} - 3)) \left[\mathbf{1} - \frac{1}{3} \mathbf{I}_{\mathbf{C}} \mathbf{C}^{-1} \right] \\ & + \sum_{kl \in S} 2a_{kl} (\mathbf{I}_{kl} - 1) \exp(b_{kl}(\mathbf{I}_{kl} - \delta_{kl})^2) \text{sym}(\mathbf{e}_k \otimes \mathbf{e}_l). \end{aligned} \quad (58)$$

The terms with $kl \in S = \{ff, ss\}$ include an approximation to the Heavyside function $\chi(\mathbf{I}_{kl}) \approx \frac{1}{1 + \exp\{-k(\mathbf{I}_{kl} - 1)\}}$. With $k = 0$, we explicitly set $\chi(\mathbf{I}_{kl}) = 1$ if $\mathbf{I}_{kl} \leq 1$, else $\chi(\mathbf{I}_{kl}) = 0$.

```

1 <Type>Holzapfel</Type>
2 <Holzapfel>
3   <a>FLOAT</a> <!-- [Pa] -->
4   <b>FLOAT</b> <!-- [Pa] -->
5   <af>FLOAT</af>

```

```

6      <bf>FLOAT</bf>
7      <as>FLOAT</as>
8      <bs>FLOAT</bs>
9      <afs>FLOAT</afs>
10     <bfs>FLOAT</bfs>
11     <kappa>FLOAT</kappa> <!-- bulk modulus [Pa] -->
12     <k>FLOAT</k> <!-- optional (default: 100) -->
13 </Holzapfel>

```

Listing 20: .xml settings for Holzapfel material

Usyk The incompressible strain energy density by Usyk et al. [4] is given by

$$\Psi^{(U)}(\mathbf{C}) = \frac{\kappa}{2}(\ln J)^2 + \frac{\mu}{2}(\exp(Q) - 1),$$

$$Q = b_{\text{ff}}E_{\text{ff}}^2 + b_{\text{ss}}E_{\text{ss}}^2 + b_{\text{nn}}E_{\text{nn}}^2 + b_{\text{fs}}(E_{\text{fs}}^2 + E_{\text{sf}}^2) + b_{\text{fn}}(E_{\text{fn}}^2 + E_{\text{nf}}^2) + b_{\text{ns}}(E_{\text{ns}}^2 + E_{\text{sn}}^2), \quad (59)$$

where $E_{ij} = \mathbf{E}\mathbf{i}_0 \cdot \mathbf{j}_0$ for $i, j \in \{f, s, n\}$ are the entries of the Green-Lagrange strain tensor $\mathbf{E} = \frac{1}{2}(\mathbf{C} - \mathbf{1})$. The 2nd Piola-Kirchhoff stress is given by

$$\mathbf{S} = \mu \exp(Q)(b \odot E)_{ij} + \kappa \ln(J)\mathbf{C}^{-1} \quad \text{with} \quad \mathbf{b} = \begin{pmatrix} b_{\text{ff}} & b_{\text{fs}} & b_{\text{fn}} \\ b_{\text{sf}} & b_{\text{ss}} & b_{\text{sn}} \\ b_{\text{nf}} & b_{\text{ns}} & b_{\text{nn}} \end{pmatrix}, \quad (60)$$

where $(A \odot B)_{ij} = (A)_{ij}(B)_{ij}$ is the Hadamard or Schur product of the matrices $\mathbf{A}$ and $\mathbf{B}$. In general, the shear stress components are assumed to be of equal contribution

$$b_{ij} = b_{ji} \quad \text{for} \quad i \neq j. \quad (61)$$

```

1      <Type>Usyk</Type>
2      <Usyk>
3          <a>FLOAT</a> <!-- shear modulus [Pa] -->
4          <bff>FLOAT</bff>
5          <bss>FLOAT</bss>
6          <bnn>FLOAT</bnn>
7          <bfs>FLOAT</bfs>
8          <bfn>FLOAT</bfn>
9          <bns>FLOAT</bns>
10         <k>FLOAT</k> <!-- bulk modulus [Pa] -->
11         <aScale>FLOAT</aScale> <!-- optional scaling for a (default 1) -->
12         <bScale>FLOAT</bScale> <!-- optional scaling for b (default 1) -->
13     </Usyk>

```

Listing 21: .xml settings for Usyk material

NeoHooke A nearly incompressible Neo-Hookean material is defined by the hyperelastic energy potential

$$\Psi^{(\text{NH})}(\mathbf{C}) = \frac{\mu}{2}(\text{tr } \bar{\mathbf{C}} - 3) + \frac{\kappa}{2}(J - 1)^2, \quad (62)$$

with the shear modulus μ , the bulk modulus κ , and the 2nd Piola-Kirchhoff stress

$$\begin{aligned}
\mathbf{S} &= \mu \frac{\partial \text{tr} \bar{\mathbf{C}}}{\partial \mathbf{C}} + \kappa \frac{d(J-1)^2}{dJ} \frac{\partial J}{\partial \mathbf{C}} \\
&= \mu \frac{\partial (\text{III}_{\mathbf{C}}^{-1/3} \mathbf{C} : \mathbb{1})}{\partial \mathbf{C}} + \kappa 2(J-1) \cdot \frac{1}{2} J \mathbf{C}^{-1} \\
&= \mu \left[\text{III}_{\mathbf{C}}^{-1/3} \mathbb{1} - \frac{1}{3} \text{III}_{\mathbf{C}}^{-1/3-1} \text{III}_{\mathbf{C}} \mathbf{C}^{-1} (\mathbf{C} : \mathbb{1}) \right] + \kappa (J-1) J \mathbf{C}^{-1} \\
&= \mu \text{III}_{\mathbf{C}}^{-1/3} (\mathbb{1} - \frac{1}{3} \mathbf{I}_{\mathbf{C}} \mathbf{C}^{-1}) + \kappa (J-1) J \mathbf{C}^{-1},
\end{aligned} \tag{63}$$

where the relationship $\frac{\partial \text{III}_{\mathbf{C}}}{\partial \mathbf{C}} = J^2 \mathbf{C}^{-1}$ was used.

```

1 <Type>NeoHooke</Type>
2 <NeoHooke>
3   <a>FLOAT</a> <!-- shear modulus [Pa] -->
4   <k>FLOAT</k> <!-- bulk modulus [Pa] -->
5 </NeoHooke>

```

Listing 22: .xml settings for NeoHooke material

5.2.2 Tension Models

File provides tensions from a tension.list file. The actual data is outsourced and managed by a FileManager object that lives within the solver class. The tension between stored data gets interpolated from a CBDataFromFile* object. You have to provide one file at location **Filename** with the following content:

```

1 # TIME # PATH to file containing the tension values for each element at
   TIME

```

Listing 23: File structure of a tension.list file

The individual files at **PATH** containing the element tension have to be binary with the following structure

```

1 int32    NrElements
2 double   tension[0]
3 double   tension[1]
4 double   tension[...]
5 double   tension[n]

```

Listing 24: File structure of individual tension files

```

1 <TensionMax> any FLOAT </TensionMax>
2 <TensionModel>File</TensionModel>
3 <File>
4   <StartTime> 0.0 </StartTime>
5   <Period> 1.0 </Period>
6   <Filename> PATH </Filename>
7 </File>

```

Listing 25: .xml settings for the File tension model

Lumens is a phenomenological model based directly on measurements of length-dependent tension in myocardial cells, as detailed by Lumens et al. [5]. This model takes into account the current length of the muscle cell, making it noteworthy for its potential to reproduce the Starling effect. The initial formulation of the model is activated by a prescribed function $F_{\text{rise}}(t)$, which computes the activation parameter C . This parameter is described as "physiologically related to intracellular calcium concentration." In contrast to the original paper, certain constants within the formula underwent conversion to ensure consistent units representing time and space, using meters (m) and seconds (s) instead of micrometers (μm) and milliseconds (ms), respectively. Additionally, for 3D simulations, C_{rest} was set to 0 to initiate the system in an equilibrium state with zero force. The equations are

$$\left\{ \begin{array}{ll} L_s = L_{s,\text{ref}} \exp(\epsilon_f), & \text{with } \epsilon_f = 1 - \lambda_f \\ F_{\text{rise}} = 0.02x^3(8 - x)^2 \exp(-x), & \text{with } x = \min(8.0, \max(t/\tau_R)) \\ \frac{dL_{sc}}{dt} = \left(\frac{L_s - L_{sc}}{L_{se,iso}} - 1 \right) v_{\text{max}}, & \\ C_L = \tanh(4 \cdot 1e12 \cdot (L_{sc} - L_{sc,0})^2), & \\ T_{\text{fun}} = \tau_{sc}(0.29 + 0.3 \cdot 1e6 \cdot L_{sc}), & \\ \frac{dC}{dt} = \frac{1}{\tau_R} C_L F_{\text{rise}} + \frac{1}{\tau_D} (C_{\text{rest}} - C) / (1 + \exp\left(\frac{T_{\text{fun}} - t}{\tau_D}\right)), & \\ t = t - t_{\text{cycle}}, \quad \text{while } t > t_{\text{cycle}} & \\ \sigma_{\text{fact}} = C(L_{sc} - L_{sc,0}) \cdot 1e6 \cdot \frac{L_s - L_{sc}}{L_{se,iso}}, & \text{if } L_{sc} \geq L_{sc,0}. \end{array} \right. \quad (64)$$

The generated tension is given by $T_a(t) = T_{\text{max}} \sigma_{\text{fact}}$.

```

1  <TensionMax> any FLOAT </TensionMax>
2  <TensionModel>Lumens</TensionModel>
3  <Lumens>
4      <startTime> 0 </startTime> <!-- [s] -->
5      <cycleLength> 0.8 </cycleLength> <!-- [s] -->
6      <Filename> Path </Filename>
7      <ExportIndices> ElementIndex </ExportIndices> <!-- export element
data to file -->
8      <Crest> 0.0 </Crest>
9      <Lsc0> 1.51 * 1e-6 </Lsc0> <!-- [m] -->
10     <Lsref> 2.0 * 1e-6 </Lsref> <!-- [m] -->
11     <Lseiso> 0.04 * 1e-6 </Lseiso> <!-- [m] -->
12     <vmax> 7.0 * 1e-6 </vmax> <!-- [1/s] -->
13     <tauD> 32.0 * 1e-3 </tauD> <!-- [s] -->
14     <tauR> 48.0 * 1e-3 </tauR> <!-- [s] -->
15     <tausc> 425.0 * 1e-3 </tausc> <!-- [s] -->
16 </Lumens>

```

Listing 26: .xml settings for the Lumens tension model

FromFunction is a collection of four simple functions to calculate an active tension response. The possibilities are **Linear**, **Sinus**, **SinusDriver**, and **DoubleHill**. Only choose one of the function types per material.

- **Linear** is a function increasing linearly between **StartTime** and **StopTime** from 0 to 1.

$$f(t) = \frac{t - t_{\text{start}}}{t_{\text{stop}} - t_{\text{start}}}. \quad (65)$$

The generated tension is given by $T_a(t) = T_{\max}f(t)$. This function can be used for constant tension at $T_{\max}$ by setting `KeepMaxValue` to `true`.

- **Sinus** is a function defining a sinus arc by using a frequency ω and phase ϕ .

$$f(t) = 0.5 - 0.5 \cos(\omega(t - t_{\text{start}}) + \phi). \quad (66)$$

The generated tension is given by $T_a(t) = T_{\max}f(t)$.

- **SinusDriver** is a sinus function for easier definition of only the positive sinus arc from `StartTime` to `StopTime`.

$$f(t) = \sin\left(\frac{\pi(t - t_{\text{start}})}{t_{\text{stop}} - t_{\text{start}}}\right). \quad (67)$$

The generated tension is given by $T_a(t) = T_{\max}f(t)$.

- **DoubleHill** is a function based on the time-dependent compliance of the left ventricle. Active tension T_a is assumed to be equal to the time course of chamber elastance $E(t)$. Stergiopoulos et al. [6] suggested that the elastance of the left ventricle can be approximated by two Hill functions:

$$e(t) = \frac{E(t) - E_{\min}}{E_{\max} - E_{\min}} = \frac{1}{k} \left(\frac{g_c}{1 + g_c} \right) \left(\frac{1}{1 + g_r} \right), \quad (68)$$

with $g_c = \left(\frac{t'}{\tau_c} \right)^{m_c}$, $g_r = \left(\frac{t'}{\tau_r} \right)^{m_r}$,

$$t' = \text{mod}(t - t_0, T), \quad k = \max(e(t)).$$

Equation (68) is used as an active stress driver function $T_a(t) = T_{\max}e(t)$. [Table 2](#) shows sample parameters for atrial and ventricular tissue.

Table 2: Parameters for the active stress driver function DoubleHill.

Description	Ventricles	Atria
Contraction rate const. m_c	1.32	1.99
Relaxation rate const. m_r	14.5	11.2
Contraction time offset τ_c	0.215 s	0.042 s
Relaxation time offset τ_r	0.362 s	0.138 s
Onset time t_0	0.15 s	0.0 s
Period T	0.8 s	0.8 s
Peak tension $T_{\max}$	80 kPa	25 kPa

```

1 <TensionMax> any FLOAT </TensionMax>
2 <TensionModel>FromFunction</TensionModel>
3 <FromFunction>
4   <!-- general settings valid for all functions -->
5   <StartTime>0.0</StartTime>
6   <StopTime>30.0</StopTime>
7   <KeepMaxValue> false </KeepMaxValue>

```



```

8      <!-- function specific options -->
9      <Type> DoubleHill </Type>
10     <DoubleHill>
11         <Period> 0.8 </Period>
12         <ContrTimeOffset> 0.215 </ContrTimeOffset>
13         <RelaxTimeOffset> 0.362 </RelaxTimeOffset>
14         <ContrRateConst> 1.32 </ContrRateConst>
15         <RelaxRateConst> 14.5 </RelaxRateConst>
16         <OnsetTime> 0.15 </OnsetTime>
17     </DoubleHill>
18
19     <Type> Sinus </Type>
20     <Sinus>
21         <Omega> 0.0 </Omega>
22         <Phi> 0.0 </Phi>
23     </Sinus>
24 </FromFunction>

```

Listing 27: .xml settings for the FromFunction tension models

Land17 implements the model by Land et al. [7] with the default parameters from the original publication. You can run this tension model in three different modes depending on your choice of the option **CalciumTransientType**:

1. with a predefined calcium transient for the ventricles (**Coppini**) or the atria (**Brixius**).
2. a calcium transient read from an **External** file (requires path to file in **CalciumFile**) sampled at 1 ms intervals.
3. coupled to an electrophysiology solver using the option **Elphy**. Currently this is only supported with *acCELLerate*, which can be used within *CardioMechanics* as a plugin (refer to [acCELLerate](#)).

By setting the option **rateDependancy** to **OFF**, stretch rate is set to $\frac{d\lambda}{dt} = 0$ at all times. Internally, the model requires the same units as in the publication. If you are unsure, refer to [7].

```

1      <TensionMax>1000</TensionMax>
2      <TensionModel>Land17</TensionModel>
3      <Land17>
4          <startTime> 0 </startTime> <!-- [s] -->
5          <cycleLength> 1.0 </cycleLength> <!-- [s] -->
6          <kff> 1.0 </kff>
7          <kss> 0 </kss>
8          <knn> 0 </knn>
9          <ksn> 0 </ksn>
10         <Filename> Path </Filename>
11         <ExportIndices> ElementIndex </ExportIndices> <!-- export element
data to file -->
12         <rateDependancy> ON </rateDependancy> <!-- ON or OFF -->
13         <CalciumTransientType> STR </CalciumTransientType> <!-- Coppini,
Brixius, External (requires CalciumFile), Elphy (coupled to
electrophysiology) -->
14         <CalciumFile> NONE </CalciumFile>
15         <TRPNn> 2.0 </TRPNn>
16         <TRPNk> 0.1 </TRPNk>

```

```

17 <Ca50> 0.805 </Ca50>
18 <ku> 1.0 </ku>
19 <nTM> 5.0 </nTM>
20 <TRPN50> 0.35 </TRPN50>
21 <kuw> 0.182 </kuw>
22 <kws> 0.012 </kws>
23 <rs> 0.25 </rs>
24 <rw> 0.5 </rw>
25 <gammas> 0.0085 </gammas>
26 <gammaw> 0.615 </gammaw>
27 <phi> 2.23 </phi>
28 <Aeff> 25.0 </Aeff>
29 <beta0> 2.3 </beta0>
30 <beta1> -2.4 </beta1>
31 <Tref> 120.0 </Tref> <!-- [kPa] -->
32 <a> 2.1 </a>
33 <k> 7.0 </k>
34 <eta_l> 200.0 </eta_l>
35 <eta_s> 20.0 </eta_s>
36 <xi> 1.0 </xi>
37 </Land17>

```

Listing 28: .xml settings for the Land17 tension model

TanH employs the phenomenological model by Niederer et al. [8] to describe the temporal development of force generation:

$$\begin{cases} T_a(t, \lambda_f) = T_{\text{peak}} \phi(\lambda) \tanh^2\left(\frac{t_s}{\tau_c}\right) \tanh^2\left(\frac{t_{\text{dur}} - t_s}{\tau_r}\right) & \text{for } 0 < t_s < t_{\text{dur}}, \\ \phi(\lambda_f) = \max\{\tanh(\text{ld}(\lambda_f - \lambda_0)), 0\}, \\ \tau_c = \tau_{c0} + \text{ld}_{\text{up}}(1 - \phi(\lambda_f)), \\ t_s = t - t_a - t_{\text{emd}}, \end{cases} \quad (69)$$

where λ_f is the fiber stretch ratio, t_a is the time of activation, T_{peak} is the peak tension, t_{emd} is the electromechanical delay, τ_c is the contraction time, τ_r is the relaxation time, ld is the degree of length dependence, ld_{up} is the length dependence upstroke time, τ_{c0} is the contraction duration, and λ_0 is the minimal fiber stretch.

```

1 <TensionMax>1000</TensionMax>
2 <TensionModel>TanH</TensionModel>
3 <TanH>
4 <startTime> 0 </startTime> <!-- [s] -->
5 <cycleLength> 1.0 </cycleLength> <!-- [s] -->
6 <kff> 1.0 </kff>
7 <kss> 0 </kss>
8 <knn> 0 </knn>
9 <ksn> 0 </ksn>
10 <Filename> Path </Filename>
11 <ExportIndices> ElementIndex </ExportIndices> <!-- export element
data to file -->
12 <MinFiberStretch> 0.7 </MinFiberStretch>
13 <EMD> 0.015 </EMD> <!-- [s] -->
14 <PeakTension> 120 </PeakTension> <!-- [kPa] -->
15 <Duration> 0.3 </Duration> <!-- [s] -->
16 <ContractionTime> 0.1 </ContractionTime> <!-- [s] -->
17 <DegreeOfLengthDependence> 5.0 </DegreeOfLengthDependence>

```

```

18     <LengthDependenceUpstrokeTime> 0.5 </LengthDependenceUpstrokeTime>
    <!-- [s] -->
19     <RelaxationTime> 0.1 </RelaxationTime> <!-- [s] -->
20 </TanH>

```

Listing 29: .xml settings for the TanH tension model

Bestel employs the phenomenological model by Bestel et al. [9], characterized through a time-dependent stress function T_a :

$$\begin{cases} \frac{dT_a(t)}{dt} = -|a(t)|T_a(t) + \sigma_0|a(t)|_+, \\ |a(t)|_+ = \max(a(t), 0), \\ a(t) = \alpha_{\max}f(t) + \alpha_{\min}(1 - f(t)), \\ f(t) = S^+(t - t_{\text{sys}})S^-(t - t_{\text{dias}}), \\ S^\pm(\Delta t) = 0.5(1 \pm \tanh(\frac{\Delta t}{\gamma})), \end{cases} \quad (70)$$

where $a(t)$ is the activation function, σ_0 is the contractility, α is the activation rate, γ is the steepness, t_{sys} and t_{dias} are the systolic and diastolic onset times, respectively.

```

1     <TensionMax>1</TensionMax>
2     <TensionModel>Bestel</TensionModel>
3     <Bestel>
4         <startTime> 0 </startTime> <!-- [s] -->
5         <cycleLength> 1.0 </cycleLength> <!-- [s] -->
6         <kff> 1.0 </kff>
7         <kss> 0 </kss>
8         <knn> 0 </knn>
9         <ksn> 0 </ksn>
10        <Filename> Path </Filename>
11        <ExportIndices> ElementIndex </ExportIndices> <!-- export element
data to file -->
12        <Contractility> 1e5 </Contractility> <!-- [Pa] -->
13        <MaxActivationRate> 5 </MaxActivationRate>
14        <MinActivationRate> -30 </MinActivationRate>
15        <Steepness> 5e-3 </Steepness>
16        <OnsetSystole> 0.17 </OnsetSystole> <!-- [s] -->
17        <OnsetDiastole> 0.484 </OnsetDiastole> <!-- [s] -->
18    </Bestel>

```

Listing 30: .xml settings for the Bestel tension model

TensionEstimator is not a model in its own right, but the counterpart of the [ActiveStressEstimator \(Inverse Problem\)](#) solver. It simply holds the tension that the solver writes into it and returns that value when the element asks for it, which makes the estimated tension act on the mechanics like any other active tension. It takes no parameters and is only meaningful together with that solver. Note that it applies the tension along the fiber direction only, without the stretch-dependent scaling used by the other tension models.

```

1     <TensionMax> 1 </TensionMax>
2     <TensionModel>TensionEstimator</TensionModel>

```

Listing 31: .xml settings for the TensionEstimator tension model

5.3 Solver

Three different solver classes are available. These are the `Static` solver, the `NewmarkBeta` solver, and the `ActiveStressEstimator` solver. The first two solve the forward problem and are described here; the `ActiveStressEstimator` solves the inverse problem and is described separately in Section 5.4. The `Static` solver does not require any further parameters to be set and is used to solve problems according to Equation 18 without momentum or problems where inertial forces can be neglected. In this case, time steps can be interpreted as loading steps instead. Dynamic problems should be solved with the `NewmarkBeta` solver, which uses the name-giving time integration method to solve the initial value problem. It provides two parameters to control the numerical stability of the solution. First order accuracy and unconditional stability can be achieved with $\beta = \frac{1}{4}(\gamma + \frac{1}{2})^2$ and $\gamma \geq \frac{1}{2}$. Using $\gamma > 0.5$ enables numerical dissipation of higher frequencies. However, it degrades the accuracy of the solver slightly. Additionally, you can choose to use a `ConsistentMassMatrix` (recommended) or a lumped mass matrix by setting the parameter to `false`. With `NewmarkBeta.Type` you can choose between the `mumps` and `superlu` solver packages to perform the factorization.

Adaptive Time Stepping Since the cardiac cycle consists of different phases that are more or less difficult to solve, CardioMechanic’s solver is equipped with an adaptive time stepping scheme. The idea behind the time stepping is to decrease the time step by a factor of two each time a simulation step fails, and to increase it by the same factor if it succeeds. Increase should happen in such a way that the same points in time are hit that would be hit by the larger time steps to ensure a consistent export of data. To prevent too many of the computationally expensive failing solver steps, the relaxation can be delayed by the following two parameters:

- `MinStep` After reduction, do at least this number of steps before trying a larger time step. Should be dividable by two.
- `FastRelaxation` Allows the time step to increase by more than one level at the same time. Tries always the maximum possible time step length after each export, no matter how small the time step was before.

The solver will adapt the time step size based on solver and solver plugin return codes:

- `CBStatus::SUCCESS`: system solved, the counter increments by one and gets subsequently checked if an increase of the step size is possible for the next solver run.
- `CBStatus::REPEAT`: system solved, but coupling condition is not fulfilled yet.
- `CBStatus::FAILED`: no solution was found with the current time step. Reduce time step by a factor of two.

In most cases, a `TimeStep` of 1 ms is sufficient to solve the mechanical system. If you do not set `MaxTimeStep`, `TimeStep` is used instead. Experience shows that if you observe more than two or three time step reductions during a simulation, it is highly likely that the simulation will fail sooner or later. Typically this happens with an insufficient quality of the tetrahedral mesh. One really bad element is all it takes to give you a lot of trouble.

```

1 <Solver>
2   <!-- General Settings -->
3   <Precision> 1e-8 </Precision>
4   <Epsilon> 1e-12 </Epsilon>
5   <IgnoreMaxIt> false </IgnoreMaxIt>
6   <MaxIterations> 50 </MaxIterations>
7   <MaxFunEval> 1000 </MaxFunEval>
8   <NonZeros> 2000 </NonZeros>
9   <ExportSNESMatrix> false </ExportSNESMatrix>
10  <StepUpIterations> 1000 </StepUpIterations>
11  <Timing> true </Timing>
12  <Formulation> TotalLagrangian </Formulation>
13
14  <!-- Set Solver -->
15  <Type> Static/NewmarkBeta/ActiveStressEstimator </Type>
16  <NewmarkBeta>
17    <!-- Dissipation of higher frequencies for Gamma >= 0.5 -->
18    <!-- Unconditional stability for 2*Beta >= Gamma >= 0.5 -->
19    <Beta> 0.25 </Beta>
20    <Gamma> 0.5 </Gamma>
21    <ConsistentMassMatrix> true </ConsistentMassMatrix>
22    <Type> mumps/superlu </Type>
23  </NewmarkBeta>
24
25  <!-- Timing -->
26  <StartTime> 0 </StartTime>
27  <StopTime> FLOAT </StopTime>
28  <TimeStep> FLOAT </TimeStep>
29  <MinTimeStep> 1e-9 </MinTimeStep>
30  <MaxTimeStep> TimeStep </MaxTimeStep>
31  <MaxInitTimeStep> MaxTimeStep </MaxInitTimeStep>
32  <FastRelaxation> false </FastRelaxation>
33  <MinSteps> 4 </MinSteps>
34 </Solver>

```

Listing 32: .xml settings for the solver class

5.4 ActiveStressEstimator (Inverse Problem)

This solver does not compute a deformation from a given active tension, but the reverse: given a target deformed surface at every point in time, it estimates the scalar active tension of every element that reproduces it. The underlying regularized Gauss-Newton formulation is described in Section 3.4. A complete, runnable setup is provided in `examples/inverseEllipsoid`.

The estimator wraps the static equilibrium solve, so it accepts the general solver parameters, but it ignores `NewmarkBeta` settings. It requires the following to be set up consistently:

- Two surfaces in the `Mesh` scope: a `CONTACT_MASTER` surface, whose nodes are compared against the target, and a `CONTACT_SLAVE` surface, whose nodes are moved onto the target.
- The `ContactHandling` plugin, which is used only to measure the distance between the two surfaces. Set `Alpha` to 0 so that no contact forces are generated.

- The `PointsCtrl` plugin (Section 5.5.3), which prescribes the target coordinates of the slave nodes over time from a list of target surface files.
- The `TensionEstimator` tension model for every material whose tension is to be estimated. It acts as a mailbox into which the solver writes the current estimate; it computes nothing on its own.

The target surfaces are usually generated from a forward simulation with the `CreateTargetSurfaces` script (Section 7.2). It also produces the list file referenced by `PointsCtrl`.

Regularization The parameters 11 to 17 are the weights λ_1 to λ_7 of Equation (33) and all default to zero. They are the main tuning knobs of the estimation and their appropriate values depend strongly on the problem, the mesh, and the noise in the target data. Since the estimation is ill-posed, at least one of them should be active; the example uses a small amount of spatial smoothing via 16 alone. For choosing them systematically, the solver writes the residual and regularization norms of every iteration to the file given by `ExportLcurveDataFile`, from which an L-curve can be constructed.

Restrictions The estimator uses a direct solve of a dense system and **runs sequentially only**. It aborts if it is started with more than one process. The size of that dense system is the number of estimated elements, which is what limits the problem size in practice; restricting the estimation to the relevant materials via `Materials` is the most effective way to keep it tractable.

```

1 <Solver>
2   <Type> ActiveStressEstimator </Type>
3   <ActiveStressEstimator>
4     <!-- Convergence of the outer iteration, at most 5 per time step
5     -->
6     <AbsTol> 1e-4 </AbsTol>
7     <RelTol> 1e-4 </RelTol>
8
9     <!-- Tikhonov weights, all default to 0 -->
10    <11> FLOAT </11> <!-- damping of the increment -->
11    <12> FLOAT </12> <!-- magnitude of the tension -->
12    <13> FLOAT </13> <!-- first temporal difference -->
13    <14> FLOAT </14> <!-- second temporal difference -->
14    <15> FLOAT </15> <!-- spatial roughness of the increment -->
15    <16> FLOAT </16> <!-- spatial roughness of the tension -->
16    <17> FLOAT </17> <!-- spatial roughness of the temporal increment
17    -->
18
19    <!-- Optional -->
20    <Materials> 30,31 </Materials>
21    <NumberOfTargetNodes> INT </NumberOfTargetNodes>
22    <ReUseInverse> false </ReUseInverse>
23    <ExportFile> PATH </ExportFile>
24    <ExportLcurveDataFile> PATH </ExportLcurveDataFile>
25  </ActiveStressEstimator>
26 </Solver>

```

Listing 33: .xml settings for the active stress estimator

The optional parameters are, in the order of the listing: `Materials` restricts the estimation to the given material indices; if it is omitted, the tension of all elements is

estimated. `NumberOfTargetNodes` subsamples the master surface to approximately this number of nodes, which reduces the cost of the sensitivity computation at the price of a coarser measurement; if it is omitted, all master nodes are used. `ReUseInverse` keeps the factorization of the tangent from the first iteration of a time step instead of rebuilding it in every iteration, which is faster but uses an increasingly outdated linearization. `ExportFile` receives the residual norm of every iteration.

5.5 Solver Plugins

The solver class supports the use of plugins which have to be activated in the `Solver` scope of the settings file. The individual plugin parameters are set in a separate `Plugins` scope, which are explained in further detail in the following sections. If you want to implement a new plugin, you should make sure and think about, which template functions defined in `CBSolverPlugin.h` you need to implement. The following is a list of the most important ones:

- `Init()`: called during solver initialization to initialize vectors, matrices, parameters etc.
- `Prepare()`: called before first `SolverStep()` to have access to the undeformed mesh.
- `Apply()`: called during `SolverStep()` before system is solved.
- `StepBack()`: controls plugin behavior when a solver step failed and is repeated with a smaller time step.
- `Export()`: primary export to VTK file called during solver export.
- `WriteToFile()`: secondary export to arbitrary files.
- `GetName()`: return the name of the plugin.
- `ApplyToNodalForces()`: to calculate and add external forces to the system, e.g. pressure.
- `AnalyzeResults()`: called during `SolverStep()` after system is solved. Can be used to check coupling conditions.
- `GetStatus()`: returns status code of the plugin at the end of `SolverStep()`.

```

1 <Solver>
2 .
3 .
4 .
5   <Plugins>
6     <Circulation>true/false</Circulation>
7     <ContactHandling>true/false</ContactHandling>
8     <LoadUnloadedState>true/false</LoadUnloadedState>
9     <acCELLerate>true/false</acCELLerate>
10    <ApplyPressure>true/false</ApplyPressure>
11    <ApplyPressureFromFunction>true/false</ApplyPressureFromFunction>
12
    <ApplyPressureFromFunctionNodeExport>true/false</ApplyPressureFromFunctionNodeExport>

```

```

13     <ReferenceRecovery>true/false</ReferenceRecovery>
14     <RobinBoundary>true/false</RobinBoundary>
15     <RobinBoundaryGeneral>true/false</RobinBoundaryGeneral>
16     <PointsCtrl>true/false</PointsCtrl>
17 </Plugins>
18 </Solver>
19
20 <Plugins>
21 ...
22 </Plugins>

```

Listing 34: .xml settings for solver plugins

5.5.1 LoadUnloadedState

The purpose of this plugin is to start a simulation using a stress and pressure free state of a geometry created from imaging. A .node file for the unloaded and inflated state is required as input for this plugin. They are typically acquired as a result of using the [ReferenceRecovery](#) plugin.

Algorithm 1 Algorithm of LoadUnloadedState plugin.

```

1: procedure LOADUNLOADEDSTATE
2:   initialize originalCoords_ = nodes_; unloadedCoords_; inflatedCoords_
3:   set nodes_ = unloadedCoords_
4:   update shape functions
5:   set velocity_ = 0; acceleration_ = 0
6:   set nodes_ = inflatedCoords_
7:   solve system once during Solver::Init()
8: end procedure

```

```

1 <LoadUnloadedState>
2   <unloadedNodes>PATH</unloadedNodes>
3   <inflatedNodes>PATH</inflatedNodes>
4 </LoadUnloadedState>

```

Listing 35: .xml settings for LoadUnloadedState plugin

5.5.2 ContactHandling

This plugin models the effect of the tissue surrounding the heart according to Fritz et al. [10], which assumes a frictionless and permanent contact between the epicardium and the pericardium using a penalty formulation. For this purpose, the so called master (Γ^M) and slave (Γ^S) surfaces are defined as a contact interface. Γ^M is defined on the epicardium and Γ^S is the inner surface of a separate volume mesh surrounding the heart. The force acting on the master surface is given by $\mathbf{f}^M = \alpha g(\mathbf{x})\mathbf{n}$ with the penalty parameter α , the gap function $g(\mathbf{x})$, and the current surface normal vector $\mathbf{n}$. For the slave surface, the same formulation is used except with negative sign for the contact forces $\mathbf{f}^S = -\mathbf{f}^M$. The gap

function $g(\mathbf{x})$ is determined by projecting a point $\mathbf{x}^m \in \Gamma^M$ onto the surface Γ^S into the direction of the current normal direction $\mathbf{n}$. Depending on the distance between $\mathbf{x}^m$ and the projected point $\mathbf{x}^s \in \Gamma^S$, the gap function reads

$$g(\mathbf{x}) = \begin{cases} \|\mathbf{x}^m - \mathbf{x}^s\|^2/2d & \text{for } \|\mathbf{x}^m - \mathbf{x}^s\| < d, \\ \|\mathbf{x}^m - \mathbf{x}^s\| - \frac{d}{2} & \text{for } \|\mathbf{x}^m - \mathbf{x}^s\| \geq d, \\ 0 & \text{for } \|\mathbf{x}^m - \mathbf{x}^s\| > d_{\max}, \end{cases} \quad (71)$$

with the `TransitionDistance` d and the `MaxDistance` $d_{\max}$ below which contact is maintained. This adaptation to the original formulation was made to yield a smooth transition in case the surfaces Γ^M and Γ^S start to overlap, which results in a change of direction of $\mathbf{n}$. Γ^M and Γ^S have to be defined under `Mesh.Surfaces` with the respective surface type of either `CONTACT_MASTER` or `CONTACT_SLAVE`. With the parameter `Alpha` you can set the penalty variable in Pa. The contact problem penalization can be ramped up during initialization to avoid large forces in the first time step. The ramp is either linear or exponential depending on the parameter `InitType` and will be carried out in n discrete `Steps`. `Beta` sets a multiplication factor for α during this phase. The option `MaxAngle` sets the maximal angle (in degree) between master and slave normals that still gets considered. The option `OneWayForce` determines if the contact force is applied in both directions (0) or if only positive (1) or negative (-1) forces are applied. If you set `SurfaceNormalDirection` as unidirectional, the forces are applied in the direction of the master surface normal. Otherwise, master surface normal and slave surface normal are added and normalized. The surface normal direction can be flipped during run time using `FlipSurfaceNormals`. If you find your slave surface is too far away from the heart's surface, you can use the option `useContactHandlingToFitPeri` to run a short simulation to slowly bring the surface normal closer to the heart. In such a simulation, it is recommended to fix the epicardial nodes using Dirichlet boundary conditions to avoid unwanted movement. If you want to apply a spatially varying penalty formulation, assign the third attribute to the master surface elements in the [Surface Elements File \(*.sur\)](#). With `Export` active, contact force, contact pressure, contact distance, and a binary state describing if a contact slave element was found are saved in arrays of the [VTK Unstructured Grid Files \(*.vtu\)](#) in each time step. Additionally, a text file with average contact distance and contact pressure is saved at `ExportPath`.

Remark: if you intent to use a combination of pericardial boundary conditions, only one plugin will export the data properly since they overwrite each others arrays. Hence, turn export only on for one of them!

```

1 <ContactHandling>
2   <Export>true/false</Export>
3   <ExportPath>PATH</ExportPath>
4   <MaxDistance>FLOAT</MaxDistance>
5   <TransitionDistance>FLOAT</TransitionDistance>
6   <MaxAngle>FLOAT</MaxAngle>
7   <Alpha>FLOAT</Alpha>
8   <Beta>FLOAT</Beta>
9   <StartTime>FLOAT</StartTime>
10  <OneWayForce>0/1/-1</OneWayForce>
11  <InitType>Linear/Exponential</InitType>
12  <Steps>INT</Steps>
13  <useContactHandlingToFitPeri>true/false</useContactHandlingToFitPeri>
14  <FlipSlaveNormals>true/false</FlipSlaveNormals>

```

```

15     <SurfaceNormalDirection>unidirectional/bidirectional</SurfaceNormalDirection>
16 </ContactHandling>

```

Listing 36: .xml settings for ContactHandling plugin

5.5.3 PointsCtrl

This plugin prescribes the coordinates of a selected set of nodes over time, read from a file. The affected nodes are fixed by a Dirichlet boundary condition and are moved to the tabulated position in every time step, so they follow the prescribed trajectory exactly instead of responding to forces. Its main use is the inverse problem (Section 5.4), where it moves the slave surface onto the measured target surface, but it can be used on its own to impose an arbitrary measured motion on part of a mesh.

The controlled nodes are given either as a file listing one node index per line (`Points.File`) or as a closed index range (`Points.FromTo`); indices are one-based in both cases. The coordinates are given by `Coordinates.File`, a list file that references one binary file of nodal coordinates per point in time, in the format described in Section 4.1.7. Values between two tabulated points in time are interpolated linearly. The order of the coordinates in those files must match the order of the controlled nodes.

```

1 <PointsCtrl>
2   <Points>
3     <!-- Either a file with one node index per line ... -->
4     <File>PATH</File>
5     <!-- ... or a range of node indices, both ends included -->
6     <FromTo>INT,INT</FromTo>
7   </Points>
8   <Coordinates>
9     <File>PATH</File>
10  </Coordinates>
11 </PointsCtrl>

```

Listing 37: .xml settings for the PointsCtrl plugin

5.5.4 Circulation

The `Circulation` plugin implements various lumped parameter circulatory system models suitable for 1-, 2-, and 4-chamber heart models. Figure 2 shows the basic class structure of this plugin. Some general settings are available for the base plugin. Chamber volumes and pressures in each time step can be exported to `IterationsFile` in CSV format. The parameter `Tolerance` decides how well 3D and 0D volumes have to match to be accepted by the solver. Surfaces to be excluded from applying pressure to can be added to `IgnoredSurfaceIndices` as a comma separated list. `PressureUnit` and `VolumeUnit` define the units used for the input parameters and model outputs. `MaxIterations` is the maximum number of iterations before the plugin return `CBStatus::FAILURE` to the solver. `Perturbation` defines, whether initial pressure perturbations shall be performed to determine the Jacobian at the beginning of each new time step. `SecantIterations` defines the number of iterations to be executed with the one-dimensional secant-method before perturbing the pressures (typically set to 0 in favor of the quasi Newton method described below).

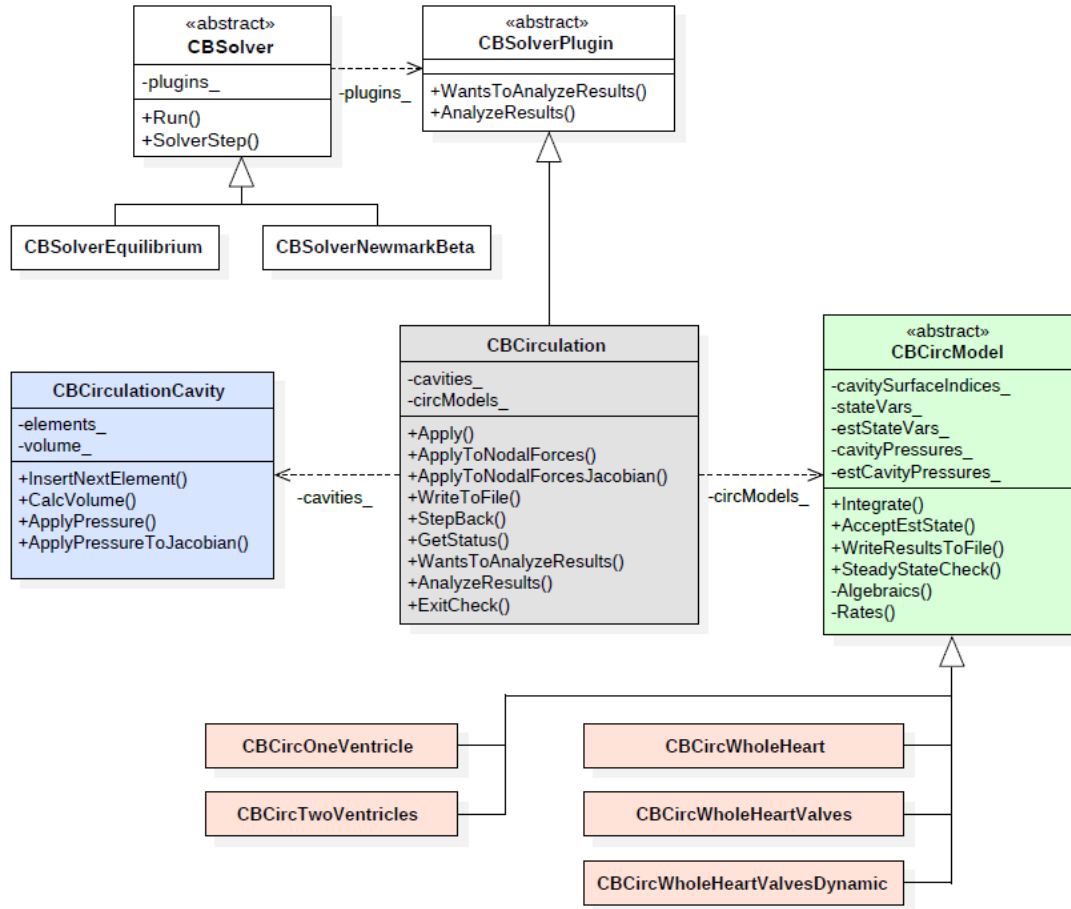


Figure 2: Simplified diagram of the circulation plugin class structure showing only the most important functions and variables. Created by Steffen Schuler.

```

1 <Circulation>
2   <IgnoredSurfaceIndices> ... </IgnoredSurfaceIndices>
3
4   <Tolerance> 1e-7 </Tolerance>
5   <MaxIterations> 250 </MaxIterations>
6   <MaxNonNewtonIterations> 10 </MaxNonNewtonIterations>
7   <Perturbation> false </Perturbation>
8   <SecantIterations> 0 </SecantIterations>
9   <IterationsFile> PATH </IterationsFile>
10
11   <PressureUnit> mmHg/hPa/Pa </PressureUnit>
12   <VolumeUnit> ml/m3 </VolumeUnit>
13   ...
14 </Circulation>

```

Listing 38: .xml settings for the circulation plugin

Coupling method The pressure values p_C with $C \in \{LV, RV, LA, RA\}$ are used as a Neumann boundary condition of the form

$$\mathbf{F} \mathbf{S} \mathbf{N} = -p_C(t) J \mathbf{F}^{-\top} \mathbf{N} \quad (72)$$

and are computed iteratively at each time step. In the first iteration $i = 0$ of each time step t_n of the mechanical system, the pressure p_C is extrapolated from the previous 5 time steps using a 4th order Adams-Bashforth scheme:

$$p_C^{n,0} = p_C^{n-1} + \Delta t_n \left(\frac{55}{24} \frac{\Delta p_C^{n-1}}{\Delta t_{n-1}} - \frac{59}{24} \frac{\Delta p_C^{n-2}}{\Delta t_{n-2}} + \frac{37}{24} \frac{\Delta p_C^{n-3}}{\Delta t_{n-3}} - \frac{9}{24} \frac{\Delta p_C^{n-4}}{\Delta t_{n-4}} \right), \quad (73)$$

with $\frac{\Delta p_C^k}{\Delta t_k} = \frac{p_C^k - p_C^{k-1}}{t_k - t_{k-1}}$, $k \in \mathbb{N}$. If there are not enough pressure values available for extrapolation (i.e. $n < 5$), the pressure is simply increased $p_C^{n,0} = p_C^{n-1} + 1$ Pa. For the sake of brevity, the four chamber pressures are now denoted $\mathbf{p}_C = (p_{LV}, p_{RV}, p_{LA}, p_{RA})^\top$. During consequent iterations $0 < i \leq 10$, the residual $\mathbf{r}_C: \mathbb{R}^4 \rightarrow \mathbb{R}^4$ for all chambers

$$\mathbf{r}_C(\mathbf{p}_C^{n,i}) = (r_{LV}, r_{RV}, r_{LA}, r_{RA})^\top,$$

with $r_C = |V_C^{3D} - V_C^{0D}|$ is used to update $\mathbf{p}_C$ by a quasi-Newton method:

$$\mathbf{p}_C^{n,i} = \mathbf{p}_C^{n,i-1} - \mathbf{C}_i^{-1} \mathbf{r}_C(\mathbf{p}_C^{n,i}), \quad (74)$$

where $\mathbf{C}_i$ is the compliance matrix determined by

$$\mathbf{C}_i^{-1} = \mathbf{C}_{i-1}^{-1} + \left(\Delta \mathbf{p}_C^{n,i} - \mathbf{C}_{i-1}^{-1} \Delta \mathbf{r}_C^{n,i} \right) \frac{\Delta (\mathbf{p}_C^{n,i})^\top \mathbf{C}_{i-1}^{-1}}{\Delta (\mathbf{p}_C^{n,i})^\top \mathbf{C}_{i-1}^{-1} \Delta \mathbf{r}_C^{n,i}}, \quad \mathbf{C}_0^{-1} = \mathbf{1},$$

with $\Delta \mathbf{p}_C^{n,i} = \mathbf{p}_C^{n,i} - \mathbf{p}_C^{n,i-1}$, $\Delta \mathbf{r}_C^{n,i} = \mathbf{r}_C(\mathbf{p}_C^{n,i}) - \mathbf{r}_C(\mathbf{p}_C^{n,i-1})$. Typically, this method satisfies the volume-consistency constraint

$$\begin{cases} V_{LV}^{3D}(\mathbf{d}(t), \mathbf{p}_C(t)) = V_{LV}^{0D}(\mathbf{c}(t), \mathbf{p}(t)) & \text{in } (0, T], \\ V_{RV}^{3D}(\mathbf{d}(t), \mathbf{p}_C(t)) = V_{RV}^{0D}(\mathbf{c}(t), \mathbf{p}(t)) & \text{in } (0, T], \\ V_{LA}^{3D}(\mathbf{d}(t), \mathbf{p}_C(t)) = V_{LA}^{0D}(\mathbf{c}(t), \mathbf{p}(t)) & \text{in } (0, T], \\ V_{RA}^{3D}(\mathbf{d}(t), \mathbf{p}_C(t)) = V_{RA}^{0D}(\mathbf{c}(t), \mathbf{p}(t)) & \text{in } (0, T], \end{cases} \quad (75)$$

with a tolerance of $\|\mathbf{r}_C\|_\infty < \varepsilon \leq 1e-7$ mL in 2 to 3 iterations. During the first time steps, the quasi-Newton update might not be sufficient in which case a standard Newton method is applied and the compliance matrix is calculated using the Jacobian $\mathbf{C} = \frac{d\mathbf{r}_C}{d\mathbf{p}_C}$ after successively perturbing one chamber at a time.

Coupling Damping When the valve model is included in the circulation model, the pressures start to oscillate with a frequency corresponding to the simulation time step. These oscillations do not decay on their own and can be explained by the effects of inertia on both sides to be coupled. This can be avoided by using the `CouplingDamping` option. First, the coupling algorithm iterates, until the coupling criterion is fulfilled, giving the pressure estimate $\mathbf{p}^n$. Then a pressure $\mathbf{p}_{\text{damped}}^n$ between this pressure and the pressure $\mathbf{p}^{n-1}$ from the previous time step is applied in one more iteration:

$$\mathbf{p}_{\text{damped}}^n = (1 - d)\mathbf{p}^n + d\mathbf{p}^{n-1}, \quad (76)$$

with the damping factor

$$d = b^n d_{\text{init}} \quad \text{with} \quad 0.5 \leq b < 1 \quad (77)$$

being the `DeclineFactor` and d_{init} is the `InitialFactor`. As soon as $d < 10^{-5}$, the coupling damping is automatically turned off. If you don't want this to happen, choose $b = 1$.

```

1   <CouplingDamping>
2       <InitialFactor> 0.5 </InitialFactor>
3       <DeclineFactor> 0.9 </DeclineFactor>
4   </CouplingDamping>

```

Listing 39: .xml settings for the coupling damping of the circulation plugin (subkeys of `<Circulation>`)

Preloading Alternatively to the [ReferenceRecovery](#) plugin, you can choose to preload the model using the `Circulation` plugin. This has the disadvantage of being less accurate and it has to be repeated with every single simulation, since no save state is produced. Nevertheless, this is how it works: (1) Gradually apply the negative diastolic pressures, which causes the chambers to shrink (set by `PreloadingTime1`). (2) Set the tension to zero again by resetting the node shape functions. Set the velocity and acceleration of nodes to zero. (3) Gradually apply the positive diastolic pressures, which inflates the chambers to approximately their original size (set by `PreloadingTime2`).

```

1   <PreloadingTime1> 0.0 </PreloadingTime1>
2   <PreloadingTime2> 0.0 </PreloadingTime2>
3   ...
4   <Circs>
5       <Circ_X>
6           ...
7           <Cavities>
8               ...
9               <!-- CircOneVentricle -->
10              <VentrPreloading> 0.0 </VentrPreloading>
11              <!-- CircTwoVentricles -->
12              <LvPreloading> 0.0 </LvPreloading>
13              <RvPreloading> 0.0 </RvPreloading>
14              <!-- 4-chamber models -->
15              <RvPreloading>0.0</RvPreloading>
16              <LvPreloading>0.0</LvPreloading>
17              <RaPreloading>0.0</RaPreloading>
18              <LaPreloading>0.0</LaPreloading>
19          </Cavities>
20      </Circ_X>
21  </Circs>

```

Listing 40: .xml settings for the preloading procedure of the circulation plugin (subkeys of `<Circulation>`)

Limit Cycle Detection A meaningful comparison of model outputs necessitates the system to be in a limit cycle. In this context, reaching a limit cycle implies a consistent blood volume distribution that remains unchanged across cardiac cycles. Consequently, pressures and flows occurring at corresponding time points of successive cycles are constant. Given the challenge in accurately estimating the simulation time required to attain a limit cycle, a limit cycle check has been incorporated. This check automatically concludes the simulation once a predefined limit cycle criterion is satisfied.

Over an extended period of time, it is imperative that both ventricles eject an equivalent volume of blood within a closed-loop circulatory system. Consequently, the parity between left ventricular (LV) and right ventricular (RV) stroke volumes serves as a robust limit cycle criterion. To ascertain the accurate disparity in stroke volumes (SVD) amid non-ideally closing valves, an additional ODE is incorporated into the circulatory model and temporally integrated:

$$\frac{dSVD}{dt} = Q_{SysArt} - Q_{PulArt} . \quad (78)$$

Starting from a designated temporal offset, the absolute value of the stroke volume difference undergoes periodic assessments (occurring at integer multiples of the cycle length) against a pre-established threshold. After each assessment, the stroke volume difference is reset to zero. This manipulation of the stroke volume difference serves as the default limit cycle criterion. Alternatively, any other output variable within a circulation model may be selected as the limit cycle criterion. The designated parameter is subsequently checked for changes in its values between two consecutive cycles, thereby assessing its temporal difference. This makes it usable for the `CircOneVentricle` model as well.

```

1      <!-- default limit cycle detection -->
2      <SteadyStateCheck>
3          <Active> true </Active>
4          <Mode> StrokeVolumeDifference </Mode>
5          <StartTime> 0.0 </StartTime>
6          <Period> 0.8 </Period>
7          <Threshold> 1.0 </Threshold>
8      </SteadyStateCheck>
9
10     <!-- alternative limit cycle detection -->
11     <SteadyStateCheck>
12         <Active> true </Active>
13         <Mode> TemporalDifference </Mode>
14         <Parameter> VentrVolume </Parameter>
15         <StartTime> 0.8 </StartTime>
16         <Period> 0.8 </Period>
17         <Threshold> 1.0 </Threshold>
18     </SteadyStateCheck>

```

Listing 41: .xml settings for a limit cycle check for all circulation models (subkey of `<Circulation><Circs><Circ_X>`)

Cavity Volume and Closed Surface Check The circulation plugin regularly requires the calculation of chamber volumes in the finite element model. The chamber volumes are calculated by summing the signed volumes of all tetrahedrons formed by the vertices $\mathbf{a}_k$, $\mathbf{b}_k$, and $\mathbf{c}_k$ of the surface triangle k and the point of origin:

$$V = \sum_k \frac{1}{6} (\mathbf{a}_k - \mathbf{0}) \cdot ((\mathbf{b}_k - \mathbf{a}_k) \times (\mathbf{c}_k - \mathbf{a}_k)) . \quad (79)$$

Equation 79 is only valid for closed surfaces. If the surface is not closed, the calculated volume generally depends on the reference point that is used to span the tetrahedrons. Therefore, each surface is checked during initialization with respect to two reference points

$\mathbf{r}_1$ and $\mathbf{r}_2$ using the following criterion:

$$|\Delta V| = \left| \sum_k \frac{1}{6} (\mathbf{a}_k - \mathbf{r}_1) \cdot ((\mathbf{b}_k - \mathbf{a}_k) \times (\mathbf{c}_k - \mathbf{a}_k)) \right| \quad (80)$$

$$- \sum_k \frac{1}{6} (\mathbf{a}_k - \mathbf{r}_2) \cdot ((\mathbf{b}_k - \mathbf{a}_k) \times (\mathbf{c}_k - \mathbf{a}_k)) \big| \stackrel{!}{<} 10^{-10} \text{mL} \quad (81)$$

CircWholeHeart/CircWholeHeartValves/CircWholeHeartValvesDynamic The 4-chamber model uses a circulatory system model first published in [11]. The systemic and pulmonary circulation are each represented by a three-element Windkessel model. With **CircWholeHeartValvesDynamic**, a description of the pressure gradient across the atrioventricular and semilunar valves is included based on Garcia et al. [12] with dynamic opening and closing of the valves [13]. Valve dynamics are replaced by an "open on pressure, close on flow" valve in **CircWholeHeartValves**. In **CircWholeHeart**, valves are represented as diodes with a characteristic resistance. For the sake of simplicity, only the equations for **CircWholeHeartValvesDynamic** are presented here.

Briefly, the model is given by

$$\frac{d\mathbf{c}(t)}{dt} - \mathbf{G}_{\mathbf{c}}(t, \mathbf{c}(t), \mathbf{p}(t)) = \mathbf{0} \quad \text{in } (0, T], \quad (82)$$

where $\mathbf{c}: (0, T] \rightarrow \mathbb{R}^{16}$ are the state variables

$$\mathbf{c} = (V_{LV}, V_{SysArt}, V_{SysVen}, V_{RA}, V_{RV}, V_{PulArt}, V_{PulVen}, V_{LA}, Q_{AV}, Q_{TV}, Q_{PV}, Q_{MV}, \eta_{AV}, \eta_{TV}, \eta_{PV}, \eta_{MV})^T,$$

associated with the chamber and vessel volumes V , the blood flow Q through the atrioventricular and semilunar valves, and the current state of the valves η . $\mathbf{G}_{\mathbf{c}}$ conveniently collects the r.h.s. of the ODEs and $\mathbf{p}: (0, T] \rightarrow \mathbb{R}^8$ contains the pressure values in all compartments of the circulatory system

$$\mathbf{p} = (p_{LV}, p_{SysArt}, p_{SysVen}, p_{RA}, p_{RV}, p_{PulArt}, p_{PulVen}, p_{LA})^T.$$

The system of ODEs is given by

$$\begin{cases} \frac{dV_{LV}}{dt} = Q_{MV} - Q_{AV}, & \frac{dV_{SysArt}}{dt} = Q_{AV} - Q_{SysPer}, \\ \frac{dV_{SysVen}}{dt} = Q_{SysPer} - Q_{SysVen}, & \frac{dV_{RA}}{dt} = Q_{SysVen} - Q_{TV}, \\ \frac{dV_{RV}}{dt} = Q_{TV} - Q_{PV}, & \frac{dV_{PulArt}}{dt} = Q_{PV} - Q_{PulPer}, \\ \frac{dV_{PulVen}}{dt} = Q_{PulPer} - Q_{PulVen}, & \frac{dV_{LA}}{dt} = Q_{PulVen} - Q_{MV}, \\ L \frac{dQ_{VT}}{dt} = \Delta p_{net} - B|Q_{VT}|Q_{VT}, & \\ \frac{d\eta_{VT}}{dt} = \begin{cases} K_o(1 - \eta_{VT})\Delta p_{net} & \text{if } \Delta p_{net} > 0, \\ K_c\eta_{VT}\Delta p_{net} & \text{if } \Delta p_{net} \leq 0, \end{cases} \end{cases} \quad (83)$$

where $B = \frac{\rho_{\text{Blood}}}{2A_{\text{Eff}}^2}$ is the Borda-Carnot resistance, $L = \frac{6.28\rho_{\text{Blood}}}{\sqrt{A_{\text{Eff}}}}$ is the inertance, ρ_{Blood} is the density of blood, and Δp_{net} is the difference in pressure across the atrioventricular and semilunar valves $VT \in \{AV, TV, PV, MV\}$. The rate of opening and closing of the valves

is given by the coefficients K_o and K_c . The effective valve area A_{Eff} depends on the state of the valve and is given by

$$A_{\text{Eff}}(t) = A_{\text{Ref}} \frac{s(t)}{1 - s(t)} \quad \text{using} \quad s(t) = (M_{\text{max}} - M_{\text{min}})\eta_{\text{VT}}(t) + M_{\text{min}}, \quad (84)$$

with the reference area A_{Ref} of the valve and the minimum and maximum area ratio, M_{min} and M_{max} , respectively. For the pulmonary and systemic circulation, the blood flow is given by

$$Q_{\text{SysPer}} = \frac{\frac{V_{\text{SysArt}}}{C_{\text{SysArt}}} - p_{\text{SysVen}}}{R_{\text{SysPer}}}, \quad Q_{\text{SysVen}} = \frac{p_{\text{SysVen}} - p_{\text{RA}}}{R_{\text{SysVen}}}, \quad (85)$$

$$Q_{\text{PulPer}} = \frac{\frac{V_{\text{PulArt}}}{C_{\text{PulArt}}} - p_{\text{PulVen}}}{R_{\text{PulPer}}}, \quad Q_{\text{PulVen}} = \frac{p_{\text{PulVen}} - p_{\text{LA}}}{R_{\text{PulVen}}}, \quad (86)$$

with the resistances R , compliances C , and the pressures defined by

$$p_{\text{SysArt}} = \frac{V_{\text{SysArt}} - V_{\text{SysArt}}^0}{C_{\text{SysArt}}} + Q_{\text{SysArt}} R_{\text{SysArt}}, \quad p_{\text{SysVen}} = \frac{V_{\text{SysVen}} - V_{\text{SysVen}}^0}{C_{\text{SysVen}}}, \quad (87)$$

$$p_{\text{PulArt}} = \frac{V_{\text{PulArt}} - V_{\text{PulArt}}^0}{C_{\text{PulArt}}} + Q_{\text{PulArt}} R_{\text{PulArt}}, \quad p_{\text{PulVen}} = \frac{V_{\text{PulVen}} - V_{\text{PulVen}}^0}{C_{\text{PulVen}}}. \quad (88)$$

Unstressed systemic and peripheral volumes are denoted by a superscript 0.

```

1 <Circulation>
2 ...
3   <Circs>
4     <Circ_1>
5       <Active>true</Active>
6       <Type>CircWholeHeartValvesDynamic</Type>
7       <ExportFile> PATH </ExportFile>
8       <MaxIntegrationTimeStep>1e-4</MaxIntegrationTimeStep>
9
10      <CircParameters>
11        <!-- CircWholeHeart -->
12        <SysArtValveResist> 0.006</SysArtValveResist>
13        <SysArtResist> 0.05 </SysArtResist>
14        <SysArtCompli> 2.5 </SysArtCompli>
15        <SysArtVolumeUnstr> 800.0 </SysArtVolumeUnstr>
16        <SysPerResist> 0.60 </SysPerResist>
17        <SysVenResist> 0.03 </SysVenResist>
18        <SysVenCompli> 100.0 </SysVenCompli>
19        <SysVenVolumeUnstr>2850.0 </SysVenVolumeUnstr>
20        <RavValveResist> 0.003</RavValveResist>
21        <PulArtValveResist> 0.003</PulArtValveResist>
22        <PulArtResist> 0.02 </PulArtResist>
23        <PulArtCompli> 10.0 </PulArtCompli>
24        <PulArtVolumeUnstr> 150.0 </PulArtVolumeUnstr>
25        <PulPerResist> 0.07 </PulPerResist>
26        <PulVenResist> 0.03 </PulVenResist>
27        <PulVenCompli> 15.0 </PulVenCompli>
28        <PulVenVolumeUnstr> 200.0 </PulVenVolumeUnstr>
29        <LavValveResist> 0.003</LavValveResist>
30        <!-- CircWholeHeartValves -->
31        <BloodDensity>7.95e-4</BloodDensity>

```



```

32     <SysArtValveMax>          0.95 </SysArtValveMax>
33     <SysArtValveMin>          0.001</SysArtValveMin>
34     <SysArtValveAreaRef>      7.0  </SysArtValveAreaRef>
35     <RavValveMax>             0.7  </RavValveMax>
36     <RavValveMin>             0.001</RavValveMin>
37     <RavValveAreaRef>         15.0  </RavValveAreaRef>
38     <PulArtValveMax>          0.95 </PulArtValveMax>
39     <PulArtValveMin>          0.001</PulArtValveMin>
40     <PulArtValveAreaRef>      7.0  </PulArtValveAreaRef>
41     <LavValveMax>             0.7  </LavValveMax>
42     <LavValveMin>             0.001</LavValveMin>
43     <LavValveAreaRef>         15.0  </LavValveAreaRef>
44     <!-- CircWholeHeartValvesDynamic -->
45     <SysArtValveRateOpening>10.0 </SysArtValveRateOpening>
46     <SysArtValveRateClosing> 6.0  </SysArtValveRateClosing>
47     <RavValveRateOpening>    20.0 </RavValveRateOpening>
48     <RavValveRateClosing>    6.0  </RavValveRateClosing>
49     <PulArtValveRateOpening>10.0 </PulArtValveRateOpening>
50     <PulArtValveRateClosing> 6.0  </PulArtValveRateClosing>
51     <LavValveRateOpening>    20.0 </LavValveRateOpening>
52     <LavValveRateClosing>    6.0  </LavValveRateClosing>
53 </CircParameters>
54 <InitialConditions>
55     <TotalVolume>            5500.0 </TotalVolume>
56     <SysArtVolume>           985.0  </SysArtVolume>
57     <PulArtVolume>           301.0  </PulArtVolume>
58     <PulVenVolume>           310.0  </PulVenVolume>
59     <SysArtFlow>              0.0    </SysArtFlow>
60     <RavFlow>                 50.0   </RavFlow>
61     <PulArtFlow>              0.0    </PulArtFlow>
62     <LavFlow>                 100.0   </LavFlow>
63     <SysArtValveState>        0.0    </SysArtValveState>
64     <RavValveState>          0.1    </RavValveState>
65     <PulArtValveState>        0.0    </PulArtValveState>
66     <LavValveState>          0.1    </LavValveState>
67     <RvPressure>              3.75   </RvPressure>
68     <LvPressure>              7.50   </LvPressure>
69     <RaPressure>              4.50   </RaPressure>
70     <LaPressure>              8.25   </LaPressure>
71 </InitialConditions>
72 <Cavities>
73     <!-- define these surfaces as CAVITY in Mesh.Surfaces -->
74     <RvSurface>130</RvSurface>
75     <LvSurface>131</LvSurface>
76     <RaSurface>132</RaSurface>
77     <LaSurface>133</LaSurface>
78 </Cavities>
79 </Circ_1>
80 </Circons>
81 </Circulation>

```

Listing 42: .xml settings for the 4-chamber circulation model

CircOneVentricle was designed for simulations with only the left ventricle. The pressure within the left ventricle is determined by a simplified closed-loop lumped parameter model of the circulatory system of the same form as Eq. (82), with the state variables $\mathbf{c}$: $(0, T] \rightarrow \mathbb{R}^3$

that are associated with the chamber and vessel volumes

$$\mathbf{c} = (V_{LV}, V_{Art}, V_{Ven})^T, \quad (89)$$

and the vector $\mathbf{p}: (0, T] \rightarrow \mathbb{R}^3$ collects the pressure values in all compartments of the system

$$\mathbf{p} = (p_{LV}, p_{Art}, p_{Ven})^T. \quad (90)$$

The system of ODEs is given by

$$\frac{dV_{LV}}{dt} = Q_{Ven} - Q_{Art}, \quad \frac{dV_{Art}}{dt} = Q_{Art} - Q_{Per}, \quad \frac{dV_{Ven}}{dt} = Q_{Per} - Q_{Ven}, \quad (91)$$

with the blood flow

$$\begin{cases} Q_{Art} = \max \left\{ \frac{p_{LV} - p_{CArt}}{R_{AV} + R_{Art}}, 0 \right\} & Q_{Per} = \frac{p_{CArt} - p_{Ven}}{R_{Per}}, \\ Q_{Ven} = \max \left\{ \frac{p_{Ven} - p_{LV}}{R_{Ven}}, 0 \right\}, \end{cases} \quad (92)$$

and the pressure

$$p_{CArt} = \frac{V_{Art} - V_{Art}^0}{C_{Art}}, \quad p_{Art} = p_{CArt} - Q_{Art} R_{Art}, \quad p_{Ven} = \frac{V_{Ven} - V_{Ven}^0}{C_{Ven}}, \quad (93)$$

in the system.

```

1 <Circulation>
2 ...
3   <Circons>
4     <Circ_1>
5       <Active>true</Active>
6       <Type>CircOneVentricle</Type>
7       <ExportFile> PATH </ExportFile>
8       <MaxIntegrationTimeStep>1e-4</MaxIntegrationTimeStep>
9
10      <CircParameters>
11        <ArtValveResist> 0.006</ArtValveResist>
12        <ArtResist> 0.07 </ArtResist>
13        <ArtCompli> 2.0 </ArtCompli>
14        <ArtVolumeUnstr> 800.0 </ArtVolumeUnstr>
15        <PerResist> 0.9 </PerResist>
16        <VenResist> 0.03 </VenResist>
17        <VenCompli> 100.0 </VenCompli>
18        <VenVolumeUnstr>2850.0 </VenVolumeUnstr>
19      </CircParameters>
20      <InitialConditions>
21        <TotalVolume> 5500.0 </TotalVolume>
22        <ArtVolume> 985.0 </ArtVolume>
23        <VentrPressure> 7.50 </VentrPressure>
24      </InitialConditions>
25      <Cavities>
26        <!-- define these surfaces as CAVITY in Mesh.Surfaces -->
27        <VentrSurface> 130 </VentrSurface>
28      </Cavities>
29    </Circ_1>
30  </Circons>
31 </Circulation>

```

Listing 43: .xml settings for the 1-chamber circulation model

CircTwoVentricles combines both ventricles in a closed circulation model by using the structure of the CircOneVentricle model twice. The state variables $\mathbf{c}: (0, T] \rightarrow \mathbb{R}^6$ that are associated with the chamber and vessel volumes

$$\mathbf{c} = (V_{LV}, V_{SysArt}, V_{SysVen}, V_{RV}, V_{PulArt}, V_{PulVen})^T, \quad (94)$$

and the vector $\mathbf{p}: (0, T] \rightarrow \mathbb{R}^6$ collects the pressure values in all compartments of the system

$$\mathbf{p} = (p_{LV}, p_{SysArt}, p_{SysVen}, p_{RV}, p_{PulArt}, p_{PulVen})^T. \quad (95)$$

The system of ODEs is given by

$$\begin{cases} \frac{dV_{LV}}{dt} = Q_{PulVen} - Q_{SysArt}, & \frac{dV_{RV}}{dt} = Q_{SysVen} - Q_{PulArt}, \\ \frac{dV_{SysArt}}{dt} = Q_{SysArt} - Q_{SysPer}, & \frac{dV_{PulArt}}{dt} = Q_{PulArt} - Q_{PulPer}, \\ \frac{dV_{SysVen}}{dt} = Q_{SysPer} - Q_{SysVen}, & \frac{dV_{PulVen}}{dt} = Q_{PulPer} - Q_{PulVen}, \end{cases} \quad (96)$$

with the blood flow

$$\begin{cases} Q_{SysArt} = \max \left\{ \frac{p_{LV} - p_{CSysArt}}{R_{AV} - R_{SysArt}}, 0 \right\}, & Q_{PulArt} = \max \left\{ \frac{p_{RV} - p_{CPulArt}}{R_{PV} - R_{PulArt}}, 0 \right\}, \\ Q_{SysVen} = \max \left\{ \frac{p_{SysVen} - p_{RV}}{R_{SysVen}}, 0 \right\}, & Q_{PulVen} = \max \left\{ \frac{p_{PulVen} - p_{LV}}{R_{PulVen}}, 0 \right\}, \\ Q_{SysPer} = \frac{p_{CSysArt} - p_{SysVen}}{R_{SysPer}}, & Q_{PulPer} = \frac{p_{CPulArt} - p_{PulVen}}{R_{PulPer}}, \end{cases} \quad (97)$$

and the pressure

$$\begin{cases} p_{CSysArt} = \frac{V_{SysArt} - V_{SysArt}^0}{C_{SysArt}}, & p_{CPulArt} = \frac{V_{PulArt} - V_{PulArt}^0}{C_{PulArt}}, \\ p_{SysVen} = \frac{V_{SysVen} - V_{SysVen}^0}{C_{SysVen}}, & p_{PulVen} = \frac{V_{PulVen} - V_{PulVen}^0}{C_{PulVen}}, \\ p_{SysArt} = p_{CSysArt} + Q_{SysArt} R_{SysArt}, & p_{PulArt} = p_{CPulArt} + Q_{PulArt} R_{PulArt}, \end{cases} \quad (98)$$

in the system.

```

1 <Circulation>
2 ...
3   <Circs>
4     <Circ_1>
5       <Active>true</Active>
6       <Type>CircTwoVentricles</Type>
7       <ExportFile> PATH </ExportFile>
8       <MaxIntegrationTimeStep>1e-4</MaxIntegrationTimeStep>
9
10      <CircParameters>
11        <SysArtValveResist> 0.006</SysArtValveResist>
12        <SysArtResist> 0.07 </SysArtResist>
13        <SysArtCompli> 2.0 </SysArtCompli>
14        <SysArtVolumeUnstr> 800.0 </SysArtVolumeUnstr>
15        <SysPerResist> 0.9 </SysPerResist>
16        <SysVenResist> 0.03 </SysVenResist>
17        <SysVenCompli> 100.0 </SysVenCompli>
18        <SysVenVolumeUnstr>2850.0 </SysVenVolumeUnstr>
19
20        <PulArtValveResist> 0.003</PulArtValveResist>
21        <PulArtResist> 0.02 </PulArtResist>
22        <PulArtCompli> 10.0 </PulArtCompli>

```

```

23         <PulArtVolumeUnstr> 150.0 </PulArtVolumeUnstr>
24         <PulPerResist> 0.07 </PulPerResist>
25         <PulVenResist> 0.03 </PulVenResist>
26         <PulVenCompli> 15.0 </PulVenCompli>
27         <PulVenVolumeUnstr> 200.0 </PulVenVolumeUnstr>
28     </CircParameters>
29     <InitialConditions>
30         <TotalVolume> 5500.0 </TotalVolume>
31         <SysArtVolume> 985.0 </SysArtVolume>
32         <PulArtVolume> 300.0 </PulArtVolume>
33         <PulVenVolume> 300.0 </PulVenVolume>
34         <LvPressure> 7.50 </LvPressure>
35         <RvPressure> 3.50 </RvPressure>
36     </InitialConditions>
37     <Cavities>
38         <!-- define these surfaces as CAVITY in Mesh.Surfaces -->
39         <LvSurface> 130 </LvSurface>
40         <RvSurface> 131 </RvSurface>
41     </Cavities>
42 </Circ_1>
43 </Circs>
44 </Circulation>

```

Listing 44: .xml settings for the 2-chamber circulation model

5.5.5 ApplyPressure

This plugin does exactly what its name suggests. It applies a pressure as a Neumann boundary condition of the form

$$\mathbf{F} \mathbf{S} \mathbf{N} = -p_C(t) J \mathbf{F}^{-T} \mathbf{N} \quad (99)$$

to a surface Γ . The pressure is calculated with

$$p(t) = p_{\max} \frac{t}{t_{\text{stop}} - t_{\text{start}}} \quad \text{if } t < t_{\text{stop}}. \quad (100)$$

If `KeepMaxPressure` is set to true, $p(t) = p_{\max} \forall t$. Pressure and volume data is exported to a file called `ApplyPressure.dat` or `Filename`.

```

1     <ApplyPressure>
2         <StartTime> 0.0 </StartTime> <!-- [s] -->
3         <StopTime> any FLOAT </StopTime> <!-- [s] -->
4         <SurfaceIndex> index </SurfaceIndex>
5         <MaxPressure> any FLOAT </MaxPressure> <!-- [Pa] -->
6         <KeepMaxPressure> true </KeepMaxPressure>
7         <Filename> STR </Filename>
8     </ApplyPressure>

```

Listing 45: .xml settings for the ApplyPressure plugin

5.5.6 ApplyPressureFromFunction

This plugin is a more sophisticated version of [ApplyPressure](#). It adds the possibilities to use different functions for the pressure calculation and the definition of multiple groups

and intervals. Pressure is calculated using

$$p(t) = p_{\text{Offset}} + p_{\text{Amp}} \cdot f(t), \quad (101)$$

where the function $f(t)$ can be defined as one of the functions given in [paragraph 5.2.2](#) by specifying one of the functions using the key **Type** in each interval. The default function is **Linear**. If the chosen function requires additional parameters, supply them with the appropriate keys. Setting the option **Invert** to true, will give the pressure

$$p(t) = p_{\text{Offset}} + p_{\text{Amp}} \cdot (1 - f(t)), \quad (102)$$

instead. Each group you define can have multiple intervals, but the intervals of each group can be different. If you define multiple intervals in a group, the **StartTime** of interval $I + 1$ has to be bigger or equal to the **StopTime** of interval I . With the option **RelaxElementsAtStart** you can recalculate the shape function derivatives and set velocity and acceleration to zero at the start of each interval. This will only affect the material given in **MaterialsToRelax**. The exported file contains pressure and volume data of each specified surface in mmHg and mL, respectively.

```

1 <ApplyPressureFromFunction>
2   <ExportFile> STR </ExportFile> <!-- default:
   ApplyPressureFromFunction.dat -->
3
4   <Groups>
5     <Group_X>
6       <Surfaces> INT, INT, ... </Surfaces>
7       <Intervals>
8         <Interval_1>
9           <Type>Linear</Type>
10          <StartTime> FLOAT </StartTime> <!-- [s] -->
11          <StopTime> FLOAT </StopTime> <!-- [s] -->
12          <Offset> FLOAT </Offset> <!-- [Pa] -->
13          <Amplitude> FLOAT </Amplitude> <!-- [Pa] -->
14          <Invert> false </Invert>
15          <RelaxElementsAtStart> false </RelaxElementsAtStart>
16          <MaterialsToRelax> INT, INT, ... </MaterialsToRelax>
17        </Interval_1>
18        <Interval_2>
19          <Type>Sinus</Type>
20          <StartTime> FLOAT </StartTime> <!-- [s] -->
21          <StopTime> FLOAT </StopTime> <!-- [s] -->
22          <Offset> FLOAT </Offset> <!-- [Pa] -->
23          <Amplitude> FLOAT </Amplitude> <!-- [Pa] -->
24          <Invert> false </Invert>
25          <RelaxElementsAtStart> false </RelaxElementsAtStart>
26          <MaterialsToRelax> INT, INT, ... </MaterialsToRelax>
27
28          <Sinus>
29            <Omega> FLOAT </Omega>
30            <Phi> FLOAT </Phi>
31          </Sinus>
32        </Interval_2>
33      </Intervals>
34    </Group_X>
35    <Group_Y>
36      <Surfaces> ... </Surfaces>

```

```

37         <Intervals>
38             <Interval_1>
39                 ...
40             </Interval_1>
41         </Intervals>
42     </Group_Y>
43 </Groups>
44 </ApplyPressureFromFunction>

```

Listing 46: .xml settings for the ApplyPressureFromFunction plugin

5.5.7 ApplyPressureFromFunctionNodeExport

Conceptually the same plugin as [ApplyPressureFromFunction](#) but with added functionality. You can either choose to terminate the simulation at a certain time (`NodeExportTime`) or when a designated closed surface (`StopSurface`) reaches the desired volume (`StopVolume`).

```

1 <ApplyPressureFromFunctionNodeExport>
2   <ExportFile> STR </ExportFile> <!-- default:
   ApplyPressureFromFunctionNodeExport.dat -->
3   <StopSurface> INT </StopSurface> <!-- default: -1 -->
4   <StopVolume> FLOAT </StopVolume> <!-- [mL] -->
5   <NodeExportTime> FLOAT </NodeExportTime> <!-- [s] -->
6   <NodeExportFile> STR </NodeExportFile>
7
8   <Groups>
9       ...
10  </Groups>
11 </ApplyPressureFromFunctionNodeExport>

```

Listing 47: .xml settings for the ApplyPressureFromFunctionNodeExport plugin

5.5.8 RobinBoundary

This plugin adds a boundary condition in the form of a traction $\mathbf{t}$ acting on the surface Γ

$$\mathbf{t} = \mathbf{N}(\alpha \mathbf{d} \cdot \mathbf{N} + \beta \dot{\mathbf{d}} \cdot \mathbf{N}). \quad (103)$$

This kind of boundary condition is typically used on the epicardium as a more simple model compared to the [ContactHandling](#) plugin. It essentially models the pericardium as a linear spring (with stiffness α) in parallel with a dashpot (with viscosity β). A more thorough explanation can be found in [14]. Compared to the [ContactHandling](#) plugin, these boundary conditions do not require the user to set up a dedicated mesh for the pericardium, thus making it more suitable for (bi-)ventricular simulations. By default, the spring stiffness and dashpot viscosity are constant on the entire surface. However, regional changes can be introduced by adding an element wise surface traction scaling to the [Surface Elements File \(*.sur\)](#). Export of this plugin adds the arrays `ContactForce`, `ContactPressure`, and `ContactDistance` to the [VTK Unstructured Grid Files \(*.vtu\)](#). The surface Γ defined by `SurfaceIndex` needs to be declared as type `CONTACT_ROBIN` using the key `Mesh.Surfaces.Surface_INT` (refer to [Mesh](#)).

Remark: if you intent to use a combination of pericardial boundary conditions, only one plugin will export the data properly since they overwrite each others arrays. Hence, turn export only on for one of them!

```

1 <RobinBoundary>
2   <Export> false </Export>
3   <StartTime> FLOAT </StartTime> <!-- [s] -->
4   <FlipSurfaceNormals> false </FlipSurfaceNormals>
5   <SurfaceIndex> INT </SurfaceIndex>
6   <alpha> FLOAT </alpha> <!-- [Pa/m] -->
7   <beta> FLOAT </beta> <!-- [(Pa*s) / m] -->
8 </RobinBoundary>

```

Listing 48: .xml settings for the RobinBoundary plugin

5.5.9 RobinBoundaryGeneral

This plugin adds a boundary condition in the form of a traction $\mathbf{t}$ acting on the surface Γ

$$\mathbf{t} = \alpha \mathbf{d} + \beta \dot{\mathbf{d}}, \quad (104)$$

i.e. it behaves like omnidirectional spring-dashpots. Otherwise, it offers the same functionality as the [RobinBoundary](#) plugin. Useful for truncated vessels, e.g. aorta and pulmonary artery.

Remark: if you intent to use a combination of pericardial boundary conditions, only one plugin will export the data properly since they overwrite each others arrays. Hence, turn export only on for one of them!

```

1 <RobinBoundaryGeneral>
2   <Export> false </Export>
3   <StartTime> FLOAT </StartTime> <!-- [s] -->
4   <FlipSurfaceNormals> false </FlipSurfaceNormals>
5   <SurfaceIndex> INT </SurfaceIndex>
6   <alpha> FLOAT </alpha> <!-- [Pa/m] -->
7   <beta> FLOAT </beta> <!-- [(Pa*s) / m] -->
8 </RobinBoundaryGeneral>

```

Listing 49: .xml settings for the RobinBoundaryGeneral plugin

5.5.10 ReferenceRecovery

A pressure and stress-free reference configuration of the heart has to be estimated to accurately model the biomechanical diastolic function. Methods applied in the context of cardiac mechanics try to find a stress-free reference configuration iteratively by using fixed-point iterations, as originally proposed by Sellier et al. [15]. To increase the convergence rate and accelerate the fixed-point method by Sellier et al., the following augmentation approach proposed by Rausch et al. [16] is used: For each iteration k , solve the forward problem $\phi(\mathbf{X}^k, p^{\text{ED}})$

$$\mathbf{f}^{\text{int}}(\mathbf{d}, T_a = 0) - \mathbf{f}^{\text{ext}}(\mathbf{d}, p^{\text{ED}}) = \mathbf{0} \quad \text{in } \Omega_k, \quad (105)$$

by applying an end-diastolic pressure p^{ED} to an intermediate configuration of the heart with coordinates $\mathbf{X}^k$ with no active stress (technically it is possible to add active stress, we just never did it). The resulting node coordinates $\mathbf{x}^k$ can be used to calculate the nodal error $\mathbf{R}^k = \mathbf{x}^k - \mathbf{x}^{\text{dat}}$ w. r. t. the target coordinates $\mathbf{x}^{\text{dat}}$. Finally, the reference coordinates are

Algorithm 2 Reference configuration recovery

```
1: procedure REFERENCERECOVERY( $\Omega_0, p^{\text{ED}}$ )
2:   initialize  $\mathbf{X}^1 = \mathbf{x}^{\text{dat}}$ ;  $k = 0$ ;  $\beta = \text{displacementFactor\_}$ ;  $n_{\text{incr}}^p = \text{numIncrements\_}$ 
3:   for  $j \leq n_{\text{incr}}^p$  do
4:      $\tilde{p} = \frac{p^{\text{ED}}}{n_{\text{incr}}^p} \cdot j$ 
5:     while  $\|\mathbf{R}^k\| \geq \text{tolerance\_}$  or  $k < \text{cycleMax\_}$  do
6:       update counter,  $k = k + 1$ 
7:       solve forward problem,  $\mathbf{x}^k = \phi(\mathbf{X}^k, \tilde{p})$ 
8:       calculate residual vector,  $\mathbf{R}^k = \mathbf{x}^k - \mathbf{x}^{\text{dat}}$ 
9:       if  $k > 1$  then
10:        update augmentation parameter,  $\beta = -\beta \frac{\mathbf{R}^{k-1} : [\mathbf{R}^k - \mathbf{R}^{k-1}]}{[\mathbf{R}^k - \mathbf{R}^{k-1}] : [\mathbf{R}^k - \mathbf{R}^{k-1}]}$ 
11:       end if
12:       update reference vector,  $\mathbf{X}^{k+1} = \mathbf{X}^k - \beta \mathbf{R}^k$ 
13:       update  $\frac{\partial N}{\partial \mathbf{X}^{k+1}}$ 
14:     end while
15:      $k = 0$ 
16:      $j++$ 
17:   end for
18:   return stress-free reference configuration,  $\mathbf{X} = \mathbf{X}^k$ 
19: end procedure
```

updated $\mathbf{X}^{k+1} = \mathbf{X}^k - \beta \mathbf{R}^k$ with the augmentation parameter β as shown in Algorithm 2. After solving the forward problem $\phi(\mathbf{X}, p^{\text{ED}})$ once again with the end-diastolic pressure values p^{ED} , the target configuration is recovered and the solution $\mathbf{d}$ together with the pressure can be used as initial conditions for further simulations:

$$\mathbf{d}^0 = \mathbf{d}, \quad p^0 = p^{\text{ED}}.$$

This method is not restricted to whole heart geometries and is applicable to (bi-)ventricular simulations as well. Sometimes the algorithm is more stable and leads to better results, if a pressure ramp is used, i.e. we divide the end-diastolic pressure into even increments and find an intermediate stress free reference configuration before we move to the next increment. Using the option `InflationDuration` combined with the solver's `TimeStep` option controls the amount of loading steps taken to from $p = 0$ Pa to $\tilde{p}$. This plugin exports four distinct files. The first file contains pressure and volume data for all surfaces. The second file contains a summary of the following data: Cycle, Increment, Time, ResidualNorm, ResidualNormL2 and UnloadedVolume. Furthermore, a [Node File \(*.node\)](#) and a [Fiber Orientation File \(*.bases\)](#) of the stress free reference state after each iteration of the algorithm is saved.

```
1 <ReferenceRecovery>
2   <ExportDir> STR </ExportDir>
3   <Surfaces> INT, INT, ... </Surfaces>
4   <Pressures> FLOAT, FLOAT, ... </Pressures> <!-- [Pa] -->
5   <NumberOfIncrements> INT </NumberOfIncrements>
6   <InflationDuration> FLOAT </InflationDuration>
7   <DisplacementFactor> FLOAT </DisplacementFactor>
8   <Precision> FLOAT </Precision>
9   <CycleMax> INT </CycleMax>
10  <Augmentation> true </Augmentation>
```


Listing 50: .xml settings for the ReferenceRecovery plugin

5.5.11 acCELLerate

This plugin makes electromechanically coupled simulations possible by solving both systems using a staggered algorithm. The class structure of this plugin and all relevant functions inside *acCELLerate* itself are shown in Figure 3. First, we shortly recapitulate how *acCELLerate* solves the monodomain equation. Using finite elements for space discretization, the monodomain equation in its semidiscretized form is given by

$$\mathbf{K}\mathbf{u}(t) = \beta C_m \mathbf{M}\dot{\mathbf{u}}(t) + \beta \mathbf{I}_{\text{ion}}(\mathbf{u}(t)) + \beta \mathbf{I}_{\text{app}}(t), \quad (106)$$

with

$$\begin{aligned} (\mathbf{K})_{ij} &= \int_{\Omega_0} (\mathbf{D}\nabla\phi_j) \cdot \nabla\phi_i \, d\Omega_0, \\ (\mathbf{M})_{ij} &= \int_{\Omega_0} \phi_j\phi_i \, d\Omega_0, \\ (\mathbf{I}_{\text{ion}}(\mathbf{u}(t)))_i &= \int_{\Omega_0} I_{\text{ion}}(v(t))\phi_i \, d\Omega_0, \\ (\mathbf{I}_{\text{app}}(t))_i &= \int_{\Omega_0} I_{\text{app}}(t)\phi_i \, d\Omega_0, \end{aligned}$$

where $\mathbf{u}$ is the transmembrane potential, β is the cell surface to volume ratio, and C_m is the membrane capacitance. To solve Equation 106 in time, *acCELLerate* uses first order Godunov splitting to decouple the parabolic diffusion problem from the ODEs of the reaction system. First, the ODEs of the reaction system are solved using explicit time integration methods such that

$$\tilde{\mathbf{u}}^{n+1} = \mathbf{u}^n + \frac{\Delta t}{C_m} [\mathbf{I}_{\text{ion}}(\mathbf{u}^n) + \mathbf{I}_{\text{app}}(n)]. \quad (107)$$

Next, we use the intermediate solution $\tilde{\mathbf{u}}^{n+1}$ to integrate the diffusion part in time by means of the Crank-Nicolson method:

$$\left(\frac{\beta C_m}{\Delta t} \mathbf{M} + \theta \mathbf{K} \right) \mathbf{u}^{n+1} = \left(\frac{\beta C_m}{\Delta t} \mathbf{M} - (1 - \theta) \mathbf{K} \right) \tilde{\mathbf{u}}^{n+1}, \quad (108)$$

using $\theta = 0.5$. The linear system (108) is then solved using the generalized minimum residual method (GMRES) with a Jacobi preconditioning to obtain $\mathbf{u}^{n+1}$.

In the case of an electromechanically coupled simulation, the stiffness matrix $\mathbf{K}$ is rebuilt after every successful mechanical time step to incorporate the deformation:

$$(\mathbf{K})_{ij} = \int_{\Omega_0} (J\mathbf{F}^{-1}\mathbf{D}\mathbf{F}^{-\top}\nabla\phi_j) \cdot \nabla\phi_i \, d\Omega_0, \quad (109)$$

using the diffusion tensor

$$\mathbf{D} = \sigma_f \frac{\mathbf{F}\mathbf{f}_0 \otimes \mathbf{F}\mathbf{f}_0}{\|\mathbf{F}\mathbf{f}_0\|^2} + \sigma_s \frac{\mathbf{F}\mathbf{s}_0 \otimes \mathbf{F}\mathbf{s}_0}{\|\mathbf{F}\mathbf{s}_0\|^2} + \sigma_n \frac{\mathbf{F}\mathbf{n}_0 \otimes \mathbf{F}\mathbf{n}_0}{\|\mathbf{F}\mathbf{n}_0\|^2}, \quad (110)$$

with the conductivities σ . Additionally, the stretch ratio λ and the stretch rate $\frac{d\lambda}{dt}$ are available to the ionic models, which in certain models (mainly Courtemanche and O'Hara-Rudy) can be used for stretch activated currents I_{sac} and a Troponin C feedback. $[\text{Ca}^{2+}]_i$ is handed to *CardioMechanics* via the function `SetActiveTensionAtQuadraturePoint()`¹, which is currently only implemented in the [Land17](#) model. Algorithm 3 shows the principle behavior of the plugin. Due to the different requirements towards mesh resolution and time integration step size, two different meshes and two different time steps can (and should) be used for electrophysiology and mechanics problems (typically $\Delta t^{\text{EP}} < \Delta t^{\text{M}}$).

Algorithm 3 Staggered coupling of *CardioMechanics* and *acCELLerate*

Require: solution of (16) at time t and $\frac{\partial N}{\partial X}$ on Ω_0^{EP}

- 1: **repeat**
 - 2: update $\mathbf{F}$, λ and $\frac{d\lambda}{dt}$ on Ω^{EP}
 - 3: do $\mathbf{X} + \Delta \mathbf{d}^t$ on Ω^{EP}
 - 4: assemble $\mathbf{K}$ using (109)
 - 5: solve Monodomain equation following (107) to (108)
 - 6: update $[\text{Ca}^{2+}]_i$ on Ω^{M}
 - 7: **until** $t + \Delta t^{\text{EP}} = t + \Delta t^{\text{M}}$
-

acCELLerate Specific Input Files Since the plugin is only an interface to an entirely different software, we need to provide an extra set of input files that are specific to the electrophysiology simulator *acCELLerate*. An overview can be found in [acCELLerate Specific Input Files](#). The files that are required to run coupled simulations are the [Config File](#) (*.actl), [Condition File](#) (*.acnd), [Material File](#) (*.def), [Cell Model File](#) (*.cmf), [CellModel Parameter File](#) (*.ev), precalculated mass and stiffness matrices in PETSc format, and the mesh as a [VTK Unstructured Grid Files](#) (*.vtu). Providing a mass and stiffness matrix is necessary for *acCELLerate* to be initialized properly. *CardioMechanics* takes advantage of this by directly copying the parallel allocation of the matrices. Both of these matrices can be set up using the tool [BidomainMatrixGenerator](#).

For efficiency reasons, the **Export** option will interpolate $[\text{Ca}^{2+}]_i$ and transmembrane voltage $\mathbf{u}$ onto the mechanical mesh.² If high resolution data on the electrophysiology mesh is required, activate export in the [Config File](#) (*.actl) or use a [Sensor File](#) (*.txt) for specific point data. With **MEF** set to **NONE**, deformation is ignored in the monodomain equation, **FULL** will follow Algorithm 3. Using the option **Material** defines the **mechanical** materials to be used for coupling. It is important that these materials exist in both the mechanical and electrophysiological mesh (even though the material number does not need to be the

¹The name of the function can be misleading. In an earlier version of the coupling algorithm, tension was calculated in *acCELLerate* and interpolated back to *CardioMechanics* (hence the name of the function). However, this resulted in instabilities when using stretch rate feedback and was therefore changed.

²Exporting on the electrophysiology mesh slowed down the simulation significantly and increased storage requirements dramatically due to the high mesh resolution. If you use the native export of *acCELLerate*, it will still slow down the simulation and increase storage requirements but not as significantly, since it will export binary PETSc vectors instead of VTK.

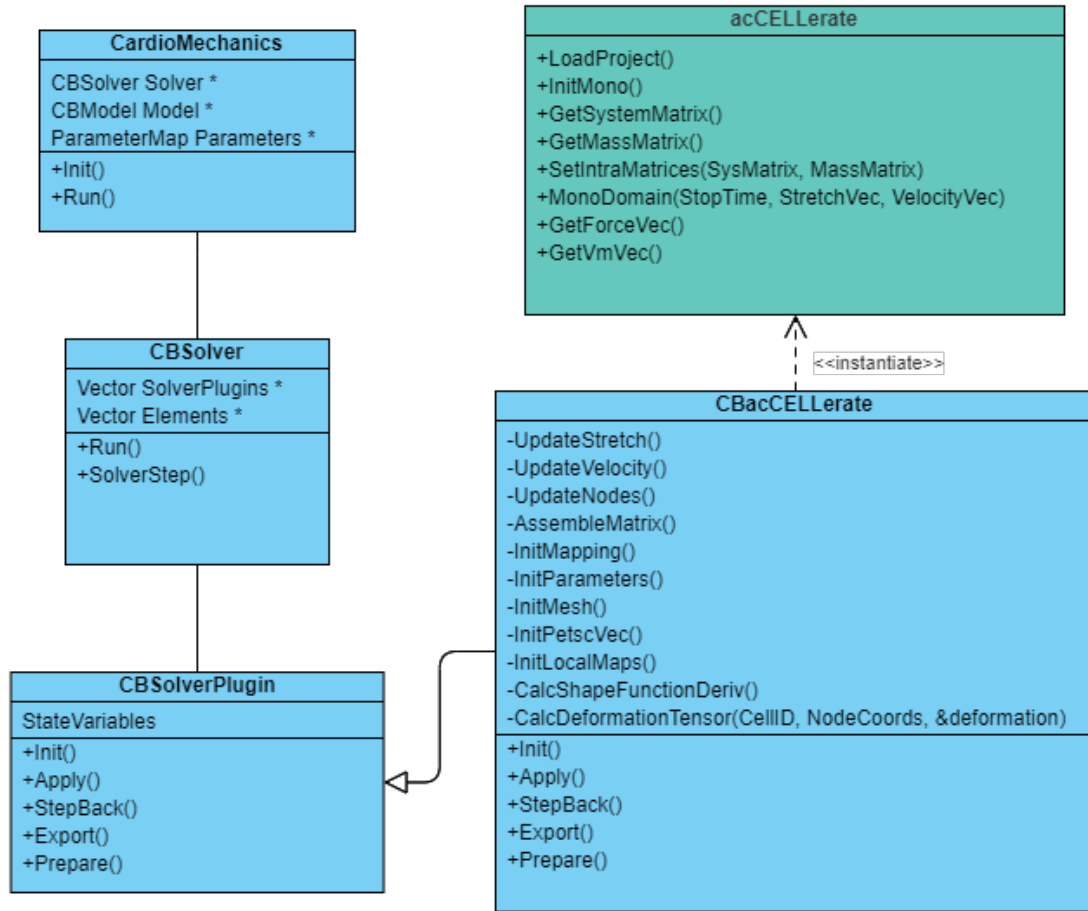


Figure 3: Class diagram of the plugin **CBacCELLerate**, which acts as an interface to the electrophysiology solver **acCELLerate**.

same). Ideally, the points of the EP mesh lie within the mechanical mesh elements or on the edges of them to ensure that the mapping algorithm works properly. In general, mesh features that exist in the EP mesh should exist in the mechanics mesh as well. This is not necessarily the case the other way around. The **INT** array specified using the option **TissuePriority** defines a custom material priority list. Materials with higher precedence should be given first. This is especially important for smaller structures such as pectinate muscles or a thin endocardial layer that might only have a one element thickness. In such a case, if the surrounding tissue would have a higher precedence, the smaller structure is not properly considered when building the matrices. By default, lower numbers are prioritized, i.e. 1, 2, 3, 4, ..., 255. Make sure to use the same tissue priority list when you allocate the matrices with **BidomainMatrixGenerator**. Supply the **Material File (*.def)** using the option **MaterialFile** and the **Config File (*.act)** using the option **ProjectFile**. The mesh for electrophysiology should be given as a **VTK Unstructured Grid Files (*.vtu)** using the option **acCELLerateMesh** and needs to contain the arrays **Material**, **Fiber**, **Sheet**, and **Sheetnormal**.

```

1 <acCELLerate>
2   <ProjectFile> STR </ProjectFile>

```

```

3 <acCELLerateMesh> STR </acCELLerateMesh>
4 <Permute> false </Permute>
5 <MaterialFile> STR </MaterialFile>
6 <TissuePriority> INT, INT, ... </TissuePriority>
7 <ResultPreFix> STR </ResultPreFix>
8 <ResultFolder> STR </ResultFolder>
9 <PvdFileName> STR </PvdFileName>
10 <Export> true </Export>
11 <MEF> NONE/FULL </MEF>
12 <Material> INT, INT, ... </Material>
13 <OffsetTime> FLOAT </OffsetTime> <!-- [s] -->
14 <constStretchRate> false </constStretchRate>
15 </acCELLerate>

```

Listing 51: .xml settings for the acCELLerate plugin

5.6 Export

Data export is defined in the [CardioMechanics Config File \(*.xml\)](#) inside the Export tag. Currently, only export as VTK files is available. Data is written to nodes and elements respectively. Some data is written to individual text files. Refer to [Table 3](#) for information about what is exported. Try to keep exported data to a minimum, since it can significantly impact simulation performance and increase file size. Plugins call their own `Export()` function which is individually controlled in the respective plugin settings.

Table 3: Export option for CardioMechanics. k indexes elements; n indexes time.

Option	File type	Location	Data exported
Active Stress	VTK	CellData	T_a
Fiber	VTK	CellData	$\mathbf{Ff}_0$
Sheet	VTK	CellData	$\mathbf{Fs}_0$
SheetNormal	VTK	CellData	$\mathbf{Fn}_0$
Jacobian	VTK	CellData	$J = \det(\mathbf{F})$
TetgenBases	VTK, .bases	CellData	$\ \mathbf{Ff}_0\ , \ \mathbf{Fs}_0\ , \ \mathbf{Fn}_0\ $ at each quadr. point
Deformation	VTK	CellData	$\mathbf{C} = \mathbf{F}^T \mathbf{F}$
DeformationEnergy	Textfile	Scalar	$E_{\text{def}} = \sum_k V_0^k \Psi^k$
ModelVolume	Textfile	Scalar	$V = \sum_k J^k / 6$
Lambda	VTK	CellData	$\sqrt{\mathbf{Ff}_0 \cdot \mathbf{Ff}_0}, \sqrt{\mathbf{Fs}_0 \cdot \mathbf{Fs}_0}, \sqrt{\mathbf{Fn}_0 \cdot \mathbf{Fn}_0}$
Cauchy	VTK	CellData	$J^{-1} \mathbf{F} \mathbf{S} \mathbf{F}^T$
PK2Stress	VTK	CellData	$\mathbf{S}$
GreenLagrangeStrain	VTK	CellData	$E = 0.5(\mathbf{C} - \mathbf{1})$
LocalActivationTime	VTK	CellData	t_a (only when using an LAT file)
KineticEnergy	Textfile	Scalar	$E_{\text{kin}} = 0.5 \mathbf{M} \mathbf{v}^2$
DampingEnergyDissipation	Textfile	Scalar	$E_{\text{damp}} = \sum_n \mathbf{C}_{\text{Rayleigh}}^n \mathbf{v}^n \mathbf{d}^n$
TotalEnergy	Textfile	Scalar	$E_{\text{tot}} = E_{\text{damp}} + E_{\text{def}} + E_{\text{kin}}$
Displacement	VTK	PointData	$\Delta \mathbf{d}^n$
Velocity	VTK	PointData	$\dot{\mathbf{d}}$
Acceleration	VTK	PointData	$\ddot{\mathbf{d}}$
AbsDisplacement	VTK	PointData	$\sum_0^n \mathbf{d}^n$
NodalForces	VTK	PointData	$f_{\text{int}} + f_{\text{ext}}$

```

1 <Export>
2   <Format>   VTK   </Format>
3   <Prefix>   prefix </Prefix>
4   <TimeStep> FLOAT </TimeStep>
5   <Options>
6     <!-- Solver export options default settings -->
7     <ActiveStress>      true </ActiveStress>
8     <Fiber>             true </Fiber>
9     <Sheet>             true </Sheet>
10    <SheetNormal>       false </SheetNormal>
11    <Jacobian>          false </Jacobian>
12    <TetgenBases>       false </TetgenBases>
13      <Deformation>     false </Deformation>
14      <DeformationEnergy> true </DeformationEnergy>
15      <ModelVolume>     true   </ModelVolume>
16    <Lambda>            true </Lambda>
17    <Cauchy>            false </Cauchy>
18    <PK2Stress>         true </PK2Stress>
19    <GreenLagrangeStrain> true </GreenLagrangeStrain>
20    <LocalActivationTime> false </LocalActivationTime>
21    <!-- Newmark export options default settings -->
22    <KineticEnergy>      false </KineticEnergy>
23    <DampingEnergyDissipation> false </DampingEnergyDissipation>
24    <TotalEnergy>        false </TotalEnergy>
25    <Displacement>       true </Displacement>
26    <Velocity>           true </Velocity>
27    <Acceleration>       true </Acceleration>
28    <AbsDisplacement>     true </AbsDisplacement>
29    <NodalForces>        false </NodalForces>
30  </Options>
31 </Export>

```

Listing 52: .xml settings for data export options.

6 Tools

In the following, you will find short descriptions of some regularly used tools that are supplied with *CardioMechanics*. Each of them can support you in either setting up your simulations or dealing with geometry related tasks. Additionally, it is recommended to take a look into the [IBT GitHub](#) for other useful tools, e.g. rule based fiber generation algorithms ([RESILIENT](#), [LDRB Fibers](#)), ventricular coordinate system ([Cobiveco](#)), and the powerful [vtkToolbox](#) for MATLAB.

6.1 BidomainMatrixGenerator

This tool generates matrices and vectors needed for mono- and bidomain FEM simulations with acCELLerate. That includes stiffness and mass matrices, materials vector, and index set. Output files are in PETSc format. When using the `-gzip` option, all output file will have the additional suffix `.gz`. You can use these files with acCELLerate without decompressing them! The only mandatory argument is the geometry as a [VTK Unstructured Grid Files \(*.vtu\)](#). It should contain the data arrays `Material` and one of the following arrays containing information about the fiber orientation:

- Phi, Theta are a specific angle measurement to define fiber orientation with a binary quantization of the angle using 8 bits or 1 Byte such that $\phi = \tilde{\phi} \cdot \frac{254}{\pi}$ and $\theta = \tilde{\theta} \cdot \frac{254}{\pi}$, where $\tilde{\phi}$ and $\tilde{\theta}$ are spherical coordinates given in radians. An orientation vector is calculated using

$$\mathbf{f}_0 = \begin{pmatrix} -\cos \tilde{\theta} \cos \tilde{\phi} \\ -\cos \tilde{\theta} \sin \tilde{\phi} \\ \sin \tilde{\theta} \end{pmatrix} \quad (111)$$

$\phi = \theta = 255$ represents the isotropic case. Aligning the fibers with the x-axis can be done with $\phi = 0$ and $\theta = 127$.

- If you already have an orientation vector, you can supply it with an array called either **Fiber** or **Orientation**.

```

1 BidomainMatrixGenerator <.vt?> [-o <prefix>] [-v]
2   [-bath <mask> [mat.def]]
3   [-mono <mat.def> [mat2.def]]
4   [-bi <intra.def> <extra.def>]
5   [-tissue <list>] [-freq <freq>]
6   [-combined] [-coupled <betaCm_dt>] [-gauss <n>]
7   [-fullLumping] [-massLumping <theta>]
8   [-iso] [-gzip] [-ignoreMissing]
9
10 -v --- Enable additional informational (verbose) output.
11
12 -bath <mask> [mat.def] --- All elements matching <mask> will only be part
    of the extracellular domain. Requires [-bi ...]. If [mat.def] is not
    given, <extra.def> is used.
13
14 -mono <mat.def> [mat2.def] --- Creates monodomain matrix. If [mat2.def] is
    given, the conductivity will be the harmonic mean of the two given
    conductivities for each cell (parallel circuit).
15
16 -bi <intra.def> <extra.def> --- Creates bidomain matrices with the intra-
    and extracellular conductivities from the given material files.
17
18 -combined --- Creates bidomain matrices where intracellular matrix is
    already added to extracellular matrix.
19
20 -coupled <betaCm_dt> [theta] --- Create fully coupled bidomain matrices
    (LHS and RHS) <betaCm_dt> is (beta*C_m)/dt. theta is the time
    discretization weight (0=explicit, 1=implicit, 0.5=Crank-Nicolson,
    default = 0).
21
22 -gauss <n> --- Create Gauss interpolation and integration matrices. <n> is
    the number of Gauss points. Default integration rules are provided.
23
24 -fullLumping --- Use full mass lumping and multiply intra/mono matrix with
    inverse of lumped mass matrix. Will not save mass matrix separately.
    Does not apply with -coupled. Use -massLumping 0 instead.
25
26 -massLumping <theta> --- Use mass lumping. Saved mass matrix will be
    weighted sum: M* = (theta)M + (1-theta)*M_lumped Usually, you would
    choose theta in [0,1]. Different choices may work, however.
27

```

```

28 -tissue <list> --- Comma-separated list of tissue classes in order of
    their, precedence when creating the tissue vector. Highest precedence
    first. Default: smaller numbers have higher precedence.
29
30 -freq <freq> --- Evaluate conductivities at frequency <freq> if supported.
31
32 -iso --- Ignore fiber orientation (isotropic case).
33
34 -ignoreMissing --- Ignore missing intra conductivities, i.e., set them to
    0. Only recommended to use with -bi.
35
36 -bds <value> --- CV stabilization factor beta*Delta_s.
37
38 -gzip --- Compress output files.
39
40 -o <prefix> --- Prefix for saving the resulting files. Defaults to the
    name of the input geometry sans extension. Depending on the given
    options, the following files may be created:
41     <prefix>.vec - Material class vector.
42     <prefix>.mat - Monodomain matrix.
43     <prefix>.mass.mat - Mass matrix.
44     <prefix>.intra.mat - Bidomain intra matrix.
45     <prefix>.extra.mat - Bidomain extra matrix.
46     <prefix>.i+e.mat - Bidomain combined matrix.
47     <prefix>.n2e.mat - Node-to-element Gauss point (interpolation)
    matrix.
48     <prefix>.e2n.mat - Element-to-node Gauss point (integration)
    matrix.
49     <prefix>.lhs.mat - Coupled LHS matrix.
50     <prefix>.rhs.mat - Coupled RHS matrix.
51     <prefix>.is - Index set of intracellular domain.

```

Listing 53: Runtime options for BidomainMatrixGenerator

6.2 PETScVec2VTK

Converts a PETSc vector as e.g. provided by acCELLerate to a VTK unstructured grid (.vtu) or VTK poly data (.vtp) file for postprocessing or visualization. The input VTK file should be the same that was used to generate the matrix using [BidomainMatrixGenerator](#). The python script [ConvertPETScDir2VTK](#) can be used to convert an entire directory at once.

```

1 PETScVec2VTK <vector filename> <VTK filename (.vtp/.vtu) in> <VTK
    filename (.vtp/.vtu) out>
2 [--arrayName] Name of the VTK array to be appended (default: Results)
3 [--delete] Delte PETScVec after conversion

```

Listing 54: Syntax for PETScVec2VTK

6.3 ConvertT4toT10

Use to convert linear elements to quadratic elements based on [Node File \(*.node\)](#) and [Element file \(*.ele\)](#).

```
1 ConvertT4toT10 <Nodes_File.node> <Elements_File.ele> <Output>
```

Listing 55: Syntax for ConvertT4toT10

6.4 VTPEExtractSurfaceFromMesh

Given a .node and .ele file of a geometry as well as a surface in .vtp/.stl format, you can use this tool to generate an appropriate .sur file with matching node indexing for use in *CardioMechanics*. The `-append` option can be used to fill a single .sur file with multiple surfaces in case of repeated execution of VTPEExtractSurfaceFromMesh.

```
1 VTPEExtractSurfaceFromMesh <nodes_file.node> <elements_file.ele> <vtp.file>
  <element_index> <surface_index> <output_prefix>
2 Options:
3   -free_surface_element_index <val>
4   -unit <val>
5   -append
6   -first_index <val>
7   -export_nodes_indices <exp_nodes_file.node>
8   -export_stand_alone <standalone_prefix>
```

Listing 56: Syntax for VTPEExtractSurfaceFromMesh

6.5 ExtractSurfaceNodesFromVTU

Extracts the deformed coordinates of all nodes belonging to the given surfaces from a simulation result and writes them in the binary format described in Section 4.1.7. It is the per-time-step building block of the target surfaces used by the inverse problem; in most cases it is not called directly but through the [CreateTargetSurfaces](#) script, which invokes it for every exported time step and additionally writes the list file. The surface indices are passed as a single quoted, space-separated argument, and the nodes are written in ascending order of their index in the .node file.

```
1 ExtractSurfaceNodesFromVTU <mesh.vtu> <nodes file> <surface file>
  '<surface indices>' <output>
2 Example:
3   ExtractSurfaceNodesFromVTU mesh.vtu mesh.node mesh.sur '1 2' surface.dat
```

Listing 57: Syntax for ExtractSurfaceNodesFromVTU

6.6 FixNodes

Can be used to apply Dirichlet boundary conditions to nodes specified in fixation.list.

```
1 FixNodes <input.node> <output.node> <fixation.list>
2   -val <val> (optional)
3   -keep_original (optional) Modify only nodes from list, if not set, all
4   other nodes are set to 0 (unfixed)
```

Listing 58: Syntax for FixNodes

7 Scripts

7.1 VTK2tetgen

Convert a vtk file to .node and .ele files for CardioMechanics. If fiber orientation is given with the appropriate Fiber, Sheet, and Sheetnormal data arrays, a .bases file is generated as well.

```
1 usage: VTK2tetgen.py [-h] [-outfile OUTFILE] [-scale SCALE]
2                     [-fixMaterial FIXMATERIAL [FIXMATERIAL ...]]
3                     mesh
4
5 positional arguments:
6   mesh                  VTK input file.
7
8 optional arguments:
9   -h, --help            show this help message and exit
10  -outfile OUTFILE       Set an alternative output filename PREFIX if
                        desired.
11  -scale SCALE           Scale the VTK input file by a factor SCALE.
12  -fixMaterial FIXMATERIAL [FIXMATERIAL ...]
13                        Fix nodes if they are connected to cells that match
14                        the given mask in "Material". Example: "-
15                        fixMaterial 32 162"
```

Listing 59: Syntax for VTK2tetgen.py

7.2 CreateTargetSurfaces

CreateTargetSurfaces.py generates the target surfaces of the inverse problem (Section 5.4) from the results of a forward simulation. For every exported VTU in -results it calls ExtractSurfaceNodesFromVTU to extract the deformed surface nodes, writes one binary file per time step into -output, and collects them in the list file given by -list, which is the file the PointsCtrl plugin expects. The time stamps in that list file are derived from -dt, which has to match the export time step of the forward run. The binary used for the extraction is taken from PATH unless -tool points at it explicitly.

```
1 usage: CreateTargetSurfaces [-h] -results RESULTS -node NODE -sur SUR
2   -output
3                               OUTPUT -list LIST -dt DT [-surfaces SURFACES]
4                               [-tool TOOL]
5 Generate inverse-problem target surfaces from a forward run.
6
7 options:
8   -h, --help            show this help message and exit
9   -results RESULTS       Directory of forward-output VTU files.
10  -node NODE             Mesh .node file.
11  -sur SUR              Mesh .sur file.
12  -output OUTPUT         Output directory for the target .dat files.
13  -list LIST            Output .list file path.
14  -dt DT               Time between exported VTUs in seconds.
15  -surfaces SURFACES     Surface indices to extract (default: '1 2').
```

```
16 -tool TOOL          Path to the ExtractSurfaceNodesFromVTU binary.
```

Listing 60: Syntax for CreateTargetSurfaces.py

7.3 ConvertPETScDir2VTK

ConvertPETScDir2VTK.py automatically converts all *.vec files in the provided directory to *.vtu format and creates a *.pvd file containing all results ready to be viewed in ParaView.

```
1 Usage: ConvertPETScDir2VTK.py [options]
2
3 Options:
4 --version          show programs version number and exit
5 -h, --help         show this help message and exit
6 -d PATH, --directory=PATH
7                     Directory containing simulation results
8 -g GEOFILE, --geometry=GEOFILE
9                     Geometry file (*.vtu) used for the simulation
10 -o FILENAME, --outfile=FILENAME
11                     Filename of the resulting *.pvd file
12 -c, --clean        Choose whether or not to delete converted *.vec
                     files
```

Listing 61: Syntax for ConvertPETScDir2VTK.py

7.4 tuneCV

This script can be used to tune conductivity values to achieve a desired conduction velocity on tetrahedral meshes based on the method described in [17]. It requires acCELLerate, Gmsh, and VTK to be installed. If your Gmsh root directory differs from the default path used on MacOS, please provide it using one of the parameters of the script. Conduction velocity is only evaluated in one direction using a planar wavefront. For anisotropic cases, you have to do multiple runs with the desired conduction velocity. Keep in mind that this evaluation is done on a very simple geometry, meaning the actual CV in your heart geometries may differ depending on a plethora of circumstances. Nevertheless, this tool should still leave you with a good estimate.

```
1 usage: tuneCV.py [-h] [-resolution RESOLUTION] [-length LENGTH]
2                  [-velocity VELOCITY]
3                  [-gi GI] [-ge GE] [-gm GM] [-beta BETA] [-cm CM] [-
4                  sourceModel {mono}]
5                  [-tol TOL] [-converge] [-maxit MAXIT] [-dt DT] [-theta
6                  {0,0.5,1}]
7                  [-stol STOL] [-lumping {True,False}]
8                  [-activationThreshold ACTIVATIONTHRESHOLD] [-log LOG]
9                  [-gmshexec GMSHEXEC]
10
11                  {BeelerReuter,TenTusscher2,CourtemancheEtAl,OHaraRudy,HimenoEtAl,KoivumaekiEtAl,GrandiEtAl}
12                  amplitude duration
13 positional arguments:
14
15                  {BeelerReuter,TenTusscher2,CourtemancheEtAl,OHaraRudy,HimenoEtAl,KoivumaekiEtAl,GrandiEtAl}
16                  Ionic model
```

```

16 amplitude           Stimulus amplitude
17 duration            Stimulus duration in s
18
19 optional arguments:
20 -h, --help           show this help message and exit
21 -resolution RESOLUTION mesh resolution in mm (default is 0.5 mm)
22 -length LENGTH       length of strand in mm (default is 20.0 mm)
23 -velocity VELOCITY   desired conduction velocity in m/s
24                     (default: 0.67 m/s)
25 -gi GI               initial value intracellular conductivity in S/m
26                     (default: 0.174 S/m)
27 -ge GE               initial value extracellular conductivity in S/m
28                     (default: 0.625 S/m)
29 -gm GM               initial value monodomain conductivity (harmonic
30                     mean of gi/ge) in S/m (default: 0.136 S/m)
31 -beta BETA           Membrane surface-to-volume ratio (default: 140000
32                     m-1)
33 -cm CM               Membrane capacitance per unit area
34                     (default: 0.01 Fm-2)
35 -sourceModel {mono} pick type of electrical source model
36                     (default is monodomain)
37 -tol TOL             Percentage of error in conduction velocity
38                     (default: 0.01)
39 -converge            Iterate until converged velocity (default: False)
40 -maxit MAXIT         Maximum number of conduction velocity iterations
41                     (default: 20)
42 -dt DT               Integration time step on seconds (default: 1e-5 s)
43 -theta {0,0.5,1}    Choose time stepping method 0: explicit, 0.5: CN,
44                     1: implicit (default: 0.5)
45 -stol STOL           Solver tolerance (default: 10-6)
46 -lumping {True,False} Use mass lumping (default: False)
47 -activationThreshold ACTIVATIONTHRESHOLD
48                     Threshold to record LAT (default: -0.02 V)
49 -log LOG             Generate/append to a log file.
50 -gmshexec GMSHEXEC  Use this option to define an alternative Gmsh path.

```

Listing 62: Syntax for tuneCV.py

7.5 CircWholeHeart

This script is an implementation of the basic 4-chamber circulatory system model described in Section 5.5.4. The script has not much use by itself, but is ready to be used with the Python library *cardioemulator* by Francesco Regazzoni to construct zero-dimensional emulators of the cardiac electromechanical function.

7.6 createSimDir

This script helps you to set up a directory for electromechanical simulations with *CardioMechanics*. All arguments are optional. However, if you already know which material numbers, conductivities, and ionic models you want to use, you can provide lists in the associated arguments. Other parameters for the monodomain equation such as membrane surface-to-volume ratio, membrane capacitance and integration time step can be given as well. This will automatically create the correct entries in the respective files. If no arguments are given, the files will be populated with some default arguments. Files that

require the geometry need to be set up by the user. The location of files that are not generated by the script are indicated by the command line output.

```

1 usage: createSimDir [-h] [-material [MATERIAL ...]]
2                   [-model
3                     [{BeelerReuter,TenTusscher2,CourtemancheEtAl,OHaraRudy,HimenoEtAl,KoivumaekiEtAl,Grandi
4                     ...}]
5                     [-sigma [SIGMA ...]] [-beta BETA] [-cm CM] [-time TIME]
6                     [-dt DT] [-theta {0,0.5,1}] [-sourceModel {mono}]
7                     [-activationThreshold ACTIVATIONTHRESHOLD] [-dir DIR]
8
9 options:
10  -h, --help            show this help message and exit
11  -material [MATERIAL ...]
12                        List of material numbers (default: 0)
13  -model
14      [{BeelerReuter,TenTusscher2,CourtemancheEtAl,OHaraRudy,HimenoEtAl,KoivumaekiEtAl,Grandi
15      ...}]
16                        List of ionic models (default: ElphyDummy)
17  -sigma [SIGMA ...]    List of monodomain conductivities in S/m (default:
18                        0.136 S/m)
19  -beta BETA            Membrane surface-to-volume ratio (default: 140000
20                        m^-1)
21  -cm CM               Membrane capacitance per unit area (default: 0.01
22                        Fm^-2)
23  -time TIME           Simulation time in s (default: 0.8 s)
24  -dt DT               Integration time step in seconds (default: 1e-5 s)
25  -theta {0,0.5,1}     Choose time stepping method 0: explicit, 0.5: CN,
26                        1:
27                        implicit (default: 0.5)
28  -sourceModel {mono}  pick type of electrical source model (default is
29                        monodomain)
30  -activationThreshold ACTIVATIONTHRESHOLD
31                        Threshold to record LAT (default: -0.02 V)
32  -dir DIR             Name of top level directory (default: SimDir)

```

Listing 63: Syntax for createSimDir.py

7.7 ClipVTUwithVTU

This script is a short post-processing tool for vtU files. Using a clipped version of the mesh created in e.g. ParaView, you can clip an entire time series while preserving all elements even when they move out of the clipped plane.

```

1 usage: ClipVTUwithVTU.py [-h] [-outfile OUTFILE] [-start START] [-stop
2   STOP] [-step STEP] mesh clip
3
4 positional arguments:
5   mesh                vtk/vtu/vtp data you want to clip.
6   clip                clipped version of <mesh> created in e.g. ParaView
7   invtu/vtk/vtp.
8
9 optional arguments:
10  -h, --help            show this help message and exit
11  -outfile OUTFILE      prefix of the results file.
12  -start START          start value for the time series.
13  -stop STOP            stop value for the time series.

```

Listing 64: Syntax for ClipVTUwithVTU.py

References

- [1] T. Gerach, “Personalized electromechanical modeling of the human heart: challenges and opportunities for the simulation of pathophysiological scenarios,” 2022.
- [2] J. M. Guccione, K. D. Costa, and A. D. McCulloch, “Finite element stress analysis of left ventricular mechanics in the beating dog heart,” *J. Biomechanics*, vol. 28, pp. 1167–1177, 1995.
- [3] G. A. Holzapfel and R. W. Ogden, “Constitutive modelling of passive myocardium: a structurally based framework for material characterization.” *Philosophical transactions. Series A, Mathematical, physical, and engineering sciences*, vol. 367, pp. 3445–75, 2009.
- [4] T. Usyk, R. Mazhari, and A. McCulloch, “Effect of laminar orthotropic myofiber architecture on regional stress and strain in the canine left ventricle,” *Journal of Elasticity*, vol. 61, pp. 143–164, 2000.
- [5] J. Lumens, T. Delhaas, B. Kirn, and T. Arts, “Three-wall segment (triseg) model describing mechanics and hemodynamics of ventricular interaction,” *Annals of biomedical engineering*, vol. 37, pp. 2234–2255, 2009.
- [6] N. Stergiopoulos et al., “Determinants of stroke volume and systolic and diastolic aortic pressure,” *American Journal of Physiology-Heart and Circulatory Physiology*, vol. 270, pp. H2050–H2059, 1996.
- [7] S. Land, S.-J. Park-Holohan, N. P. Smith, C. G. Dos Remedios, J. C. Kentish, and S. A. Niederer, “A model of cardiac contraction based on novel measurements of tension development in human cardiomyocytes,” *Journal of Molecular and Cellular Cardiology*, vol. 106, pp. 68–83, 2017.
- [8] S. A. Niederer, G. Plank, P. Chinchapatnam, et al., “Length-dependent tension in the failing heart and the efficacy of cardiac resynchronization therapy.” *Cardiovascular research*, vol. 89, pp. 336–43, 2011.
- [9] J. Bestel, F. Clément, and M. Sorine, “A biomechanical model of muscle contraction,” in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2001*, W. J. Niessen and M. A. Viergever, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1159–1161.
- [10] T. Fritz, C. Wieners, G. Seemann, H. Steen, and O. Dössel, “Simulation of the contraction of the ventricles in a human heart model including atria and pericardium : Finite element analysis of a frictionless contact problem,” *Biomechanics and Modeling in Mechanobiology*, vol. 13, pp. 627–641, 2014.
- [11] T. Gerach et al., “Electro-mechanical whole-heart digital twins: A fully coupled multi-physics approach,” *Mathematics*, vol. 9, p. 1247, 2021.

- [12] D. Garcia et al., “Analytical modeling of the instantaneous pressure gradient across the aortic valve,” *J Biomech*, vol. 38, pp. 1303–11, 2005.
- [13] J. P. Mynard et al., “A simple, versatile valve model for use in lumped parameter and one-dimensional cardiovascular models,” *International Journal for Numerical Methods in Biomedical Engineering*, vol. 28, pp. 626–641, 2012.
- [14] M. R. Pfaller, J. M. Hörmann, M. Weigl, et al., “The importance of the pericardium for cardiac biomechanics: from physiology to computational modeling,” *Biomechanics and modeling in mechanobiology*, vol. 18, pp. 503–529, 2019.
- [15] M. Sellier, “An iterative method for the inverse elasto-static problem,” *Journal of Fluids and Structures*, vol. 27, pp. 1461–1470, 2011.
- [16] M. K. Rausch, M. Genet, and J. D. Humphrey, “An augmented iterative method for identifying a stress-free reference configuration in image-based biomechanical modeling,” *Journal of Biomechanics*, vol. 58, pp. 227–231, 2017.
- [17] C. M. Costa, E. Hoetzel, B. M. Rocha, A. J. Prassl, and G. Plank, “Automatic parameterization strategy for cardiac electrophysiology simulations,” in *Computing in Cardiology 2013*. IEEE, 2013, pp. 373–376.